Manuscript submitted for review; publication dates are assigned by the journal.

Digital Object Identifier to be assigned by the journal

STRATA: A Fixed-Buffer Scoped Symbol Table with LIFO Undo and Bounded Probing

ADHYAN JAIN[1] and SHUBHI SINGH[1], and BHUVANESHWARI M.[1]
[1]School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu 632014, India

Corresponding author: Bhuvaneshwari M. (e-mail: m.bhuvaneswari@vit.ac.in). Other e-mail addresses: adhyan.jain2024@vitstudent.ac.in (A. Jain), shubhi.singh2024@vitstudent.ac.in (S. Singh).

This work received no external funding. The authors declare no competing interests.

ABSTRACT A compiler that runs inside a fixed memory budget needs a symbol table whose size is fixed in advance, whose behavior at the limit is defined, and whose index work per call can be bounded on hostile source text. We present STRATA, a scoped symbol table that keeps an open-addressing index, packed identifier records and an undo stack in one caller-supplied buffer; a call that does not fit is refused and leaves the buffer unchanged. For every open-addressing rule that never moves a key, removing the newest key by clearing its cell restores the previous index exactly, and for linear probing last-in-first-out order is the only deletion order safe for every table. In an ideal random-hash model this gives the exact probability that a probe limit refuses the next fresh name; for a table of 65 536 slots at load three quarters a refusal probability of one in a million per request needs 258 probes, against 455 for a tail bound. The law agrees with the implementation in 2.8 million trials, and we bound the refusal-filtered process that the law does not describe exactly. On events logged by a C compiler for eight projects STRATA gave no wrong answer and needed a fixed buffer 16.4 to 17.9 percent smaller than a conventional single-buffer table that stores a hash word, but only 5.3 to 9.0 percent smaller than one that does not, a saving that comes from name packing. In repeated timing sessions it was slower than both conventional tables, by 8 to 20 and up to 13 percent. Bounded mode limits index slot examinations per declaration and lookup, not scope exit or the default-mode rebuild. Identifiers that collide for every seed slow the unkeyed table 300-fold; a keyed hash removes this effect for that collision family.

INDEX TERMS Compilers, embedded software, hash tables, linear probing, memory management, symbol tables.

I. INTRODUCTION

A SCOPED symbol table maps identifiers to declarations and forgets the declarations of a block when the block ends [1]. It is one of the oldest data structures in compiler construction [2], and for a compiler on a workstation it is a solved problem: a hash table from the standard library and an undo stack are enough [3]. The situation changes when the compiler itself is the constrained program. A compiler or interpreter front end that runs on a microcontroller, inside a boot loader, or in a tool that must never exceed a fixed allocation has three requirements that the standard solution does not meet.

Bounded memory. The table must live in storage whose size is fixed before the first declaration, without calls to an allocator whose fragmentation and failure behavior are outside the compiler's control.

Defined behavior at the bound. When a declaration does not fit, the table must say so and remain usable. A table that is left half-updated turns an ''out of memory'' diagnostic into an incorrect compilation.

Bounded work on untrusted input. If the source text is supplied by someone else, the cost of one table operation must not depend on that person's choice of identifiers. Hash tables are a known target of algorithmic complexity attacks [4], and seeding a common non-cryptographic hash does not prevent them [5].

The scoped tables we found in textbooks and in production compilers [3], [6], [7] obtain memory from an allocator, do not define their state after a failed allocation, and use unkeyed hashes. On the analytical side, the theory of open addressing covers insertions and deletions in arbitrary order [8]–[10]; the deletion order of a scoped table, last in first out, has not been treated as a case of its own, and probe limits are chosen with tail bounds whose looseness is unknown.

This article asks what it costs to meet all three requirements and how to configure a table that meets them, and answers

by building, measuring and analyzing. The object of study is STRATA, a scoped symbol table in one buffer of 32-bit words. Its algorithm is the textbook one: one hash index, an undo stack, restoration on scope exit [3]. Our contributions lie in what is built around that algorithm, in what can be proved about it, and in how it is evaluated.

1) **A single-buffer scoped symbol table with an all-or-nothing refusal.** Index, interned names with their innermost declaration, hidden declarations and scope marks share one caller-supplied buffer. An operation that does not fit returns a status code and leaves the state, as tested byte for byte, unchanged (Sections IV and VII).
2) **Exact undo and its converse.** For every open-addressing scheme that never moves a key, removing the newest key by clearing its cell restores exactly the previous index, so the index always equals the insertion-only index of the live names. For a given table the safe deletion orders are exactly the linear extensions of a dependency relation (for greedy probing), and last-in-first-out order is the only one safe for every table (proved for linear probing; Section V).
3) **An exact refusal law for the probe limit.** With a static index and a limit of L probes per call, the probability that the next legitimate name is refused is computed exactly at finite size in the ideal hashing model for the first capped query, validated against the implementation, bounded for the refusal-filtered process, and turned into design values; the Chernoff-type bound used before is shown to be loose by a factor growing like $L^{3/2}$ (Section VI).
4) **A measured decomposition on compiler-generated workloads.** We separate what is gained by keeping everything in one allocation-free buffer, which a conventional chained table also achieves, from what the compact design adds, and an exact space ledger attributes the latter to a stored hash word and name packing rather than to interning. We also report that an apparent time penalty of the compact design was an implementation artifact (Section IX).
5) **An attack, its defenses, and a key-lifetime rule.** We construct C identifiers that collide in the default hash for every seed, measure the slowdown and the cost of a keyed hash and the probe limit, and state the refusal law for chosen names under the assumption of a fresh key per table and an oblivious adversary (Sections VI-C and IX-H).

We do not claim a new hashing scheme, a faster symbol table, or a result on hardware. The compact table is not faster than the conventional one; the refusal law is classical as a quantity and is credited as such, and the new content is its use, validation and the bounds for the refusal-filtered process; the collision construction applies a published technique [5]; and the microcontroller is emulated. Section XII lists what the evidence does not cover.

Section II reviews related work and Section III states the problem. Section IV describes the design, Section V the exact undo, and Section VI the bounded mode and its refusal law. Section VII describes how the guarantees were checked, Sections VIII and IX give the experimental setup and results, and Section X the interactive tool. Sections XI to XIII discuss, list limitations and conclude.

II. RELATED WORK

A. SCOPED SYMBOL TABLES

Graham *et al.* [2] described hashed symbol tables for languages with explicit scope control. The scheme used here is the one Appel presents for block-structured languages [3]: a single hash table maps each name to its innermost binding, an auxiliary stack records the names bound in each scope, and leaving a scope pops the stack and restores what was hidden. Lookup cost is independent of the nesting depth, in contrast to a stack of per-scope maps [1]. LLVM's `ScopedHashTable` [6] is a reusable implementation with nodes obtained from an allocator; lcc [11] links one table per scope level, and the Tiny C Compiler [7] keeps symbol stacks that are popped at scope exit. All of these allocate per symbol or per node, none defines the state of the table after an allocation fails, and none bounds the work of a call. Region allocation [12] removes most of the allocator's cost for compilers; a chained table whose entries live in a stack-like arena is its natural single-buffer form and is our main baseline. Microcontroller runtimes such as MicroPython intern identifiers in pools [13], which addresses the cost of names but not scopes or failure behavior.

B. DELETION IN OPEN ADDRESSING

Removing a key from a linear-probing table normally requires a tombstone or a backward shift [8], [14], and the way deletions are implemented strongly affects linear probing over time [9]. Sanders [15] gives the test for when a single deleted element must become a tombstone. Robin Hood hashing [16], used by the predecessor of this work [17], and cuckoo and hopscotch hashing [18], [19] move keys. That the most recently inserted key of linear probing can be removed by emptying its slot follows from the insertion rule and is likely folklore in backtracking code; what we add is the statement for every non-reordering scheme, the characterization of all safe orders, and their use for exact analysis. The property is related to, but weaker than, history independence [20], [21]: here the layout is a function of the live names and their order.

C. ANALYSIS OF OPEN ADDRESSING

For an ideal hash, Knuth analyzes insertion-only linear probing exactly [8]; the distribution of the unsuccessful-search cost, whose tail is the refusal probability studied here, is known through exact generating functions and Poisson limits [22]–[24], and the displacement law of Robin Hood hashing [25], [26] reappears in Section V as a law of orphaned keys. Recent work determines optimal probe complexity for open addressing without reordering [10] and the cost

of dynamic workloads [27]. Alharbi, Banderier and Bouillaguet [28] analyze bounded linear probing, in which a key that exceeds the probe cap is discarded and redrawn; our event, refusal with no change, under last-in-first-out deletions, is a different process. Pagh *et al.* [29] and Thorup [30] analyze linear probing under limited independence.

D. ALGORITHMIC COMPLEXITY ATTACKS

Crosby and Wallach [4] showed that an attacker who chooses the keys can degrade a hash table to a list. Aumasson and Bernstein [5] showed that MurmurHash2 and MurmurHash3 admit multicollisions that hold for every seed and proposed SipHash; the accompanying presentation [31] gives the construction for MurmurHash3. HalfSipHash with one compression and three finalization rounds is documented in the Linux kernel as suitable only for mitigating hash-table flooding [32]. Adaptive attacks through timing are an old observation [33], [34]. Compilers for trusted input commonly choose speed instead: the hash inside the Rust compiler is deliberately unkeyed [35]. We take the opposite case and measure both the attack and the defenses on a scoped table.

E. SUMMARY OF DIFFERENCES

Table 1 summarizes the properties relevant here. Entries for other systems are findings from the public documentation and source code that we read, at the versions we consulted; they are not a claim about every release. ''No'' means that we found no such mechanism. In the refusal column it does not distinguish an allocation policy that aborts on failure, which is defined but fatal, from an undefined partially updated state; we did not separate the two.

III. PROBLEM FORMULATION

A. OPERATIONS AND SEMANTICS

A scoped symbol table supports four operations on a stack of scopes that initially holds one outermost scope.

- `enter`: push a new innermost scope.
- `exit`: remove the innermost scope and all declarations made in it; refused at the outermost scope.
- `declare`(x, v): bind name x to value v in the innermost scope. If x is already bound in that scope the value is replaced; otherwise a new declaration is created, which hides any declaration of x in an enclosing scope.
- `resolve`(x): return the value of the innermost visible declaration of x, or report that there is none.

A name is a nonempty byte string of at most 255 bytes. A value is a 32-bit integer with an 8-bit type tag. We write d for the number of live declarations, $n \leq d$ for the number of distinct names among them, s for the number of open scopes besides the outermost, and $w(x)$ for the number of 32-bit words that name x occupies.

B. REQUIREMENTS

R1 **Fixed storage.** All state resides in one buffer of C words supplied by the caller. No allocator is called.

R2 **All-or-nothing refusal.** Every operation either succeeds or returns a status code and leaves the complete state unchanged.

R3 **Correct scoping.** Every `resolve` returns what the definition above prescribes, given the operations that succeeded.

R4 **Bounded index work per call.** In static-index (bounded) mode, a constant L, independent of d and of the names chosen, bounds the number of index slots examined by one `declare` or `resolve`. This is a bound on slot examinations, not on the total work of every call: `exit` takes time proportional to the declarations of the scope it closes, and in the default mode a declaration that grows the index rebuilds it in time proportional to n. Section VI translates L into a total-work bound.

R5 **Rare, quantified refusal of honest input.** Under R4, the probability that a declaration is refused although memory is available must be known and negligible for names that were not chosen to collide.

R1 to R3 are met by the default mode of STRATA and by the baseline. R4 and R5 are met by the bounded mode, R5 under the assumptions of Section VI.

C. THREAT MODEL

The adversary supplies the source text and thereby every name and the order of all operations. The adversary knows the implementation, including the hash function, but not a 64-bit key that the device draws from a source the adversary cannot predict. We separate two adversaries. An *oblivious* adversary fixes all names before the key is drawn; an *adaptive* one chooses names after observing the table's behavior, for instance through refusals or timing, within one table. The goal is to make table operations slow or, with a probe limit, to have legitimate declarations refused. We do not consider memory-safety attacks on the surrounding compiler or side channels through which the key might leak; Section VI-C states what an adversary who can observe probe lengths gains. Exhausting the buffer with many distinct names is always possible and is not an attack on the table: it is the budget doing its job, and R2 keeps the table usable afterwards.

IV. DESIGN OF STRATA

A. ARCHITECTURE AND LAYOUT

Fig. 1 shows the system and Fig. 2 the layout. All state lives in one array of 32-bit words. The *index* occupies the first m words, m a power of two. A slot is zero when empty; otherwise it holds an 8-bit fingerprint taken from the hash and a 24-bit reference to an identifier record. Collisions are resolved by linear probing, and the index is kept at most three quarters full.

The *identifier records* follow the index and grow upward, one per distinct visible name. A record holds the value of the name's *innermost* declaration, a word of metadata (number of name words, an encoding flag, the type tag, and the depth of the scope that made the innermost declaration), and the name. A lookup therefore ends in the record it finds, and its

TABLE 1. Properties of Scoped Symbol Tables and Identifier Stores Relevant to This Work.

Design	Scopes with shadowing	One caller-supplied buffer	Defined refusal, state unchanged	No rebuild inside a call	Bound on probes, exact refusal law	Keyed hash
Undo-stack scheme [3], LLVM `ScopedHashTable` [6]	yes	no	no	no	no	no
Tiny C Compiler symbol stacks [7]	yes	no	no	no	no	no
MicroPython interned strings [13]	no	pools	not stated	n/a	no	no
METIS-X [17] (predecessor)	yes	no	no	no	no	no
Conventional arena table (our baseline)	yes	yes	yes	no	no	no
STRATA, default mode	yes	yes	yes	no	no	optional
STRATA, bounded mode	yes	yes	yes	yes	yes	optional

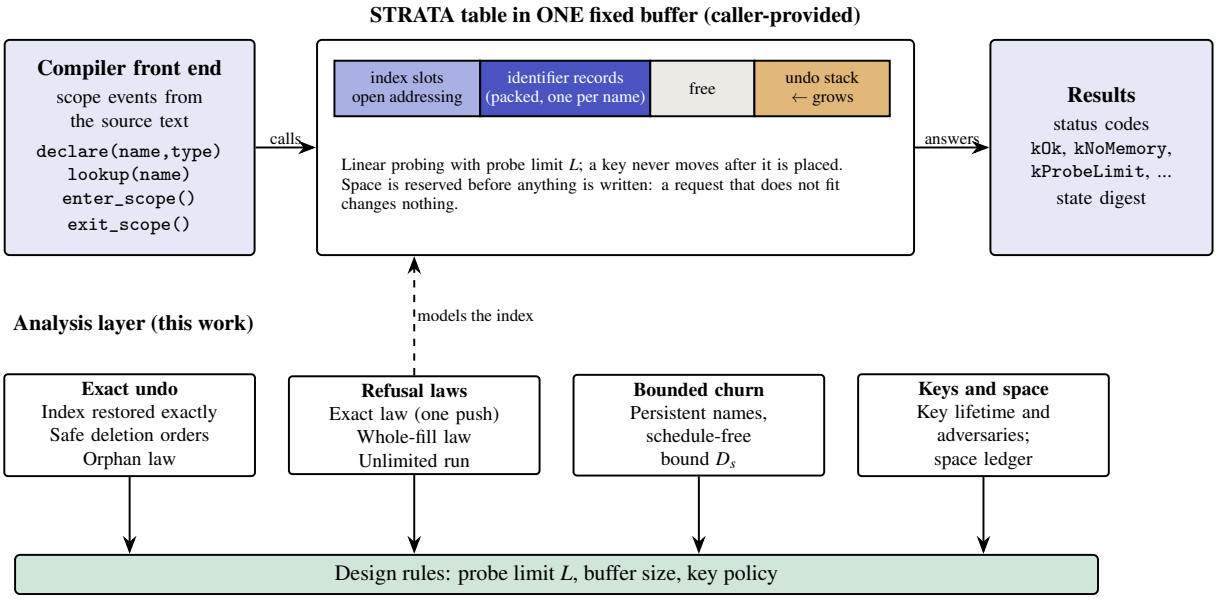


FIGURE 1. Architecture. The compiler front end issues four calls to a STRATA table that keeps its open-addressing index, packed identifier records and undo stack in one fixed buffer; the analysis of Sections V and VI turns the model of the index into design rules for the probe limit, the buffer size and the key policy.

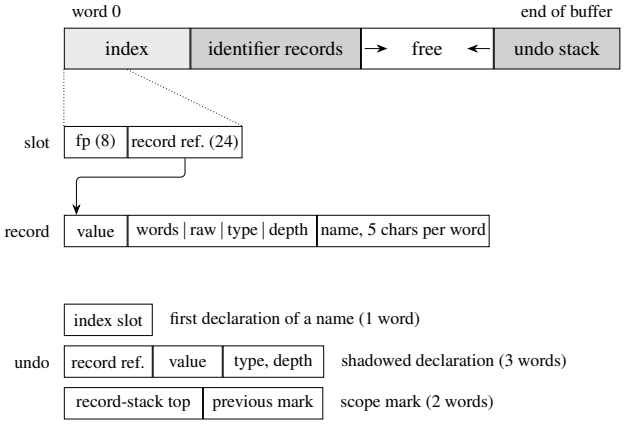


FIGURE 2. Layout of the buffer and of the three kinds of undo entries. The index and the identifier records grow from the start of the buffer, the undo stack from its end; the free space between them is shared.

TABLE 2. Bit-Level Encoding of the Buffer (32-Bit Words, Bit 0 Least Significant). References Count Words.

Item	Fields
Index slot (1 word)	bits 31–24 fingerprint (top byte of the hash, 1 if 0); 23–0 identifier reference; 0 = empty
Identifier record	word 0 value; word 1 meta; then $n \leq 64$ name words
Meta word	6–0 name words n; 7 raw flag; 9–8 raw length mod 4; 17–10 type tag; 31–18 scope depth ($\leq 16\,383$)
Undo, new name (1 word)	index slot, $< 2^{24}$, so bit 24 is clear
Undo, shadowing (3 words)	word 0 record reference with bit 24 set; word 1 saved value; word 2 saved type and depth (bits 31–10 of meta)
Scope mark (2 words)	word 0 identifier-stack top at entry; word 1 position of the previous mark

cost does not depend on how often the name is shadowed. Names consisting only of letters, digits and the underscore are packed five characters to a word, six bits each, with the codes 1 to 63 (zero is never a character code); the query is packed once and compared word by word. Any other name is stored as raw bytes, four to a word. Table 2 gives every field.

Packed names are unambiguous without a length field: a character code is never zero, so zero padding in the last

word cannot be read as a character, and two packed names of different lengths differ in the number of words or in the padded positions of the last word. Raw names are copied byte by byte, padded with zero bytes, and keep their length modulo 4 in the meta word, so names that end in or contain zero bytes remain distinct; the raw flag separates raw from packed names of equal words. Words are stored in host byte order, so a buffer is not portable between hosts of different endianness, and the keyed hash was checked against the reference code only on a little-endian host. References are 24 bits wide and reference zero is reserved (*itop* starts at 1), so the largest buffer that can be used is $2^{24} - 1$ words, 67 108 860 bytes, 4 bytes short of 64 MiB. The identifier stack cannot reach the end of the reference range before the buffer is full, so the explicit reference check of Algorithm 1 is a defensive guard that never fired in our tests. The buffer must be 4-byte aligned; the library does not check this, and it is part of the caller's contract.

The *undo stack* grows downward from the end of the buffer. A declaration of a name that was not visible pushes one word, the index slot of the new record. A declaration that hides an outer one pushes three words: the record reference and the value, type and depth that were overwritten. Entering a scope pushes a two-word mark holding the top of the record stack and the position of the previous mark. With m index slots the buffer is used up to

$$M = m + 1 + \sum_{x} \big(2 + w(x)\big) + n + 3\,(d - n) + 2\,s \quad (1)$$

words, where the sum runs over the n distinct visible names and the constant accounts for the reserved reference zero. The conventional table of Section VIII-B stores, for every declaration, a chain link, the hash, the value, a length-and-depth word and the unpacked name.

B. DECLARATION AND LOOKUP

Algorithm 1 gives the declaration as implemented. It has the form *validate, search, reserve, write*: the search is read-only, the single operation that can change the buffer before the last refusal point is Ensure (Algorithm 3), and that operation either completes or leaves the state unchanged. A failed search ends at the first empty slot on the name's probe path, which is exactly where linear probing would insert the name; the slot is remembered and reused, so a declaration walks the path once. If Ensure rebuilt the index, which cannot happen in bounded mode, the remembered slot is stale and the name is reinserted instead.

Status precedence. The checks run in the order of Algorithm 1: an invalid table, an empty name, an over-long name, then `redeclared` (which needs no reservation), then `no-memory` (reference range, load limit, space), and `probe-limit` last. A name that is both too long and unaffordable reports `name-too-long`; one that is unaffordable and would also exceed the probe limit reports `no-memory`. Scope entry checks, in order, validity, nesting depth (`too-deep`) and space; scope exit checks validity

Algorithm 1 Declare(x, v)

Input: name x (1 to 255 bytes), value and type v; current depth δ; probe limit L (bounded mode) or $L = \infty$
Output: a status; the state is unchanged unless the status is `ok` or `redeclared`

1: **if** table invalid **then**
2: **return** `no-memory`
3: **end if**
4: $K \leftarrow$ MakeKey(x); **if** $|x| = 0$ **return** `empty-name`; **if** $|x| > 255$ **return** `name-too-long`
5: $(r, e) \leftarrow$ Find(K, L) {read-only; record or first empty slot within L probes}
6: **if** $r \neq$ `nil` **and** $depth(r) = \delta$ **then**
7: overwrite value and type of r;
8: **return** `redeclared`
9: **else if** $r \neq$ `nil` **then**
10: **if not** Ensure$(3, 0)$ **then**
11: **return** `no-memory`
12: **end if**
13: push $(r \mid$ shadow bit, $value(r)$, $type(r)$, $depth(r))$; overwrite value, type, depth of r;
14: **return** `ok`
15: **end if**
16: **if** $itop + 2 + |K| > 2^{24} - 1$ **then**
17: **return** `no-memory`
18: **end if**
19: **if not** Ensure$(3 + |K|, 1)$ **then**
20: **return** `no-memory`
21: **end if** {may grow; all-or-nothing}
22: **if** $L < \infty$ **and** $e =$ `nil` **then**
23: **return** `probe-limit`
24: **end if** {last refusal point}
25: append the record for x with v and δ; store reference and fingerprint in e (or reinsert, if the index was rebuilt)
26: push the slot on the undo stack;
27: **return** `ok`

and then `no-scope`. The text name is copied into a stack-resident key (268 bytes: 64 name words and three words of header) before any mutation, so a name that points into the table's own buffer is read completely before a growable buffer moves. Reservations are computed in 64-bit arithmetic, and the 24-bit limits on references, name length (255 bytes), words per name (64) and depth (16 383) are checked before the corresponding write.

A lookup packs and hashes the name and probes from its home slot until it finds a slot whose fingerprint, metadata and name words match, or an empty slot, or, in bounded mode, until L slots have been examined. The fingerprint is a filter only; a match is always confirmed by comparing the name.

C. SCOPE ENTRY AND EXIT WITHOUT TOMBSTONES

Entering a scope reserves two words and pushes a mark; it can be refused only for depth or space. Leaving a scope pops the undo stack down to the scope's mark, newest entry first

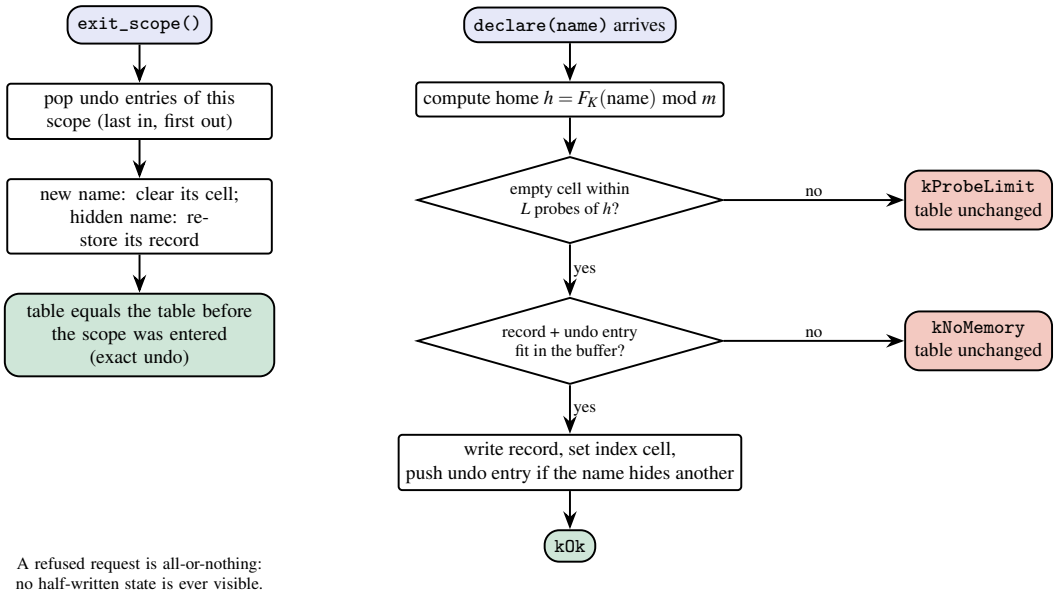


FIGURE 3. Flow of a declaration and of a scope exit. A declaration is refused, with the table unchanged, when no empty slot is found within *L* probes or when the record and its undo entry do not fit; a scope exit pops the undo entries in reverse order.

(Algorithm 2, Fig. 3). A three-word entry restores the value, type and depth it saved. A one-word entry empties the index slot it names. Then the top of the record stack is reset to the value in the mark, which discards the records created in the scope. Emptying a slot is sufficient because of the order of removal: the key being removed is the newest key in the index, so it lies on no other key's probe path. Section V proves this for every non-reordering scheme and shows that no other order has the property. No tombstone is written and no key is shifted. Scope exit cannot be refused for lack of memory: it frees storage and, in the default mode, may then give memory back, which is optional and leaves the table valid if the allocator declines.

D. ALL-OR-NOTHING REFUSAL

Each operation validates its arguments, reserves everything it needs, and only then writes. In fixed mode the reservation is a comparison of counters. When the index must grow, Ensure (Algorithm 3) checks that the larger buffer exists before it moves anything: the only step that can fail is obtaining the new block, and it comes first. After a successful reservation no step can fail. Consequently every refusal (no memory, nesting deeper than 16 383, an empty or over-long name, and the probe limit) leaves the state exactly as it was; Section VII describes the tests of this for every reservation boundary.

E. GROWING THE INDEX

In the default mode the index doubles when it would exceed its load limit. The identifier records are moved up as one block, which their relative references permit, and the index is rebuilt by reinserting the names oldest first, so that the property used in Section IV-C holds again. One-word undo entries name index slots, and those change. So that each record finds its undo entry again without a search and without scratch memory, which a fixed buffer does not have, each one-word entry and its record exchange one word before the rebuild and swap back during reinsertion. Every name is hashed once, and the remembered insertion slot of a declaration is discarded because the index changed. When only a growable buffer is extended, the index and the records do not move and nothing is rehashed. In the default mode a single call that triggers a rebuild takes time proportional to n; the bounded mode removes this case.

V. EXACT UNDO AND SAFE DELETION ORDERS

Model the index as a ring of m cells. A rule is *non-reordering* if a key never moves after it is placed and the cell chosen for an insertion depends only on the set of occupied cells and on the key's own probe sequence; this covers linear, quadratic and double hashing and uniform probing, and excludes cuckoo, hopscotch and Robin Hood hashing. The operations are *push*, which inserts a name that is not live, and *pop*, which removes the most recently pushed live name by clearing its cell; a refused push changes nothing. In STRATA the pushes are the *first* declarations of names, since a shadowing declaration needs no index slot. For a sequence $a = (a_1, \ldots, a_t)$ of distinct names, $\mathrm{Ins}(a)$ is the table obtained by inserting them in this order into the empty table.

Theorem 1 (Exact undo): For every non-reordering rule, after any sequence of pushes and pops, with refused pushes leaving the state unchanged, the table equals $\mathrm{Ins}(a)$, where a is the live sequence in push order.

Proof: Induction on the number of operations. A refused push changes nothing. A successful push of x onto $\mathrm{Ins}(a)$

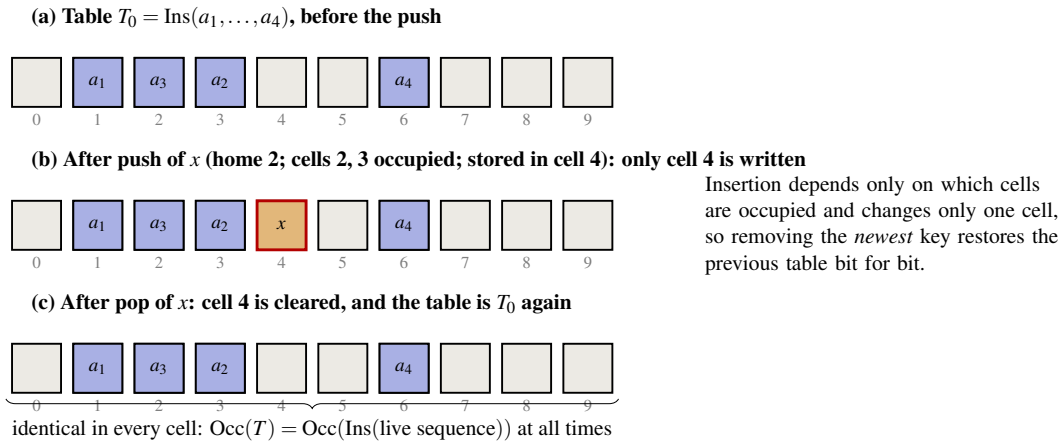


FIGURE 4. **Exact undo. Pushing x writes one empty cell; popping x clears it, and the table is identical in every cell to the table before the push.**

Algorithm 2 Enter and Exit

procedure Enter
if table invalid **then**
  return `no-memory`
end if
if $\delta \geq 16\,383$ **then**
  return `too-deep`
end if
if not Ensure$(2, 0)$ **then**
  return `no-memory`
end if {last refusal point}
push mark $(itop, mark)$; $mark \leftarrow$ its position; $\delta \leftarrow \delta + 1$;
return `ok`
procedure Exit
if table invalid **then**
  return `no-memory`
end if
if $mark = 0$ **then**
  return `no-scope`
end if {last refusal point}
while undo stack above $mark$ **do**
  $\epsilon \leftarrow$ top entry
  if ϵ is a shadow entry **then**
    restore value, type, depth of its record; pop 3 words
  else
    empty the index slot ϵ; pop 1 word
  end if
end while
$(itop, mark) \leftarrow$ saved fields of the mark; pop 2 words; $\delta \leftarrow \delta - 1$;
return `ok`

Algorithm 3 Ensure(w, a): room for w stack words and a new index entries

if static index (bounded mode) **then**
  return $(\text{names} + a) \cdot 4 \leq \text{slots} \cdot \lambda$ **and** free words $\geq w$
end if {pure comparison}
$m' \leftarrow m$; **while** $(\text{names} + a) \cdot 4 > m'\lambda$ **do** $m' \leftarrow 2m'$ {$\lambda = 3$ (or 2, sparse)}
if $m' = m$ **and** free words $\geq w$ **then**
  return true
end if
$need \leftarrow m' + itop + undo + w$
if $need \leq C$ **then**
  return Relocate(C, m')
end if {index grows inside the buffer}
if fixed buffer **then**
  return false
end if
$C' \leftarrow \min(1.5\,C$ or $need$, limits); **if** $C' < need$ **return** false
return Relocate(C', m')
procedure Relocate(C', m') {the only mutating step}
obtain the new block (the single failure point; on failure return false); stash undo entries; move undo stack, then records; rebuild the index oldest first; update C, m, undo references; return true

writes x into the cell that the rule chooses from the occupied set, which gives $\mathrm{Ins}(a{\cdot}x)$ by definition. For a pop, $a = u{\cdot}x$ and the table is $\mathrm{Ins}(u)$ with x additionally in the cell its insertion chose, because an insertion changes exactly one cell and that cell was empty in $\mathrm{Ins}(u)$; clearing it gives $\mathrm{Ins}(u)$ (Fig. 4). ■

The proof uses neither the lookup rule nor randomness, so it holds for adversarial names. What it proves is a statement about the *index cells*: the set of occupied cells and their contents equal those of $\mathrm{Ins}(a)$. For STRATA the claim carries over to the bytes of the buffer only for the state that the operations define, and we state exactly which. We distinguish four levels, each tested over 66 582 scope exits in eleven configurations (fixed, bounded and growable buffers, odd sizes, tiny buffers that refuse). (a) *Logical state* (every accessor, the lookup of every name, the invariants): equal to a fresh

table of the live declarations in all exits. (b) *Allocated region* (index, identifier stack, undo stack): equal to a fresh table with the same index size in all exits; the identifier stack alone is equal even when the index size differs, because references are relative, whereas the index words and one-word undo entries differ across index sizes by construction. In growable mode the index is not shrunk at once, so after a scope exit it can be larger than that of a fresh table built from the live names, and equality with such a table holds only when the index is configured to the same size. (c) *Every supplied word*: never equal, because removal is by moving counters: dead records, popped undo entries and scope marks remain in the free gap between the two stacks, and the high-water statistics are history. Only the removed name's index slot is cleared. (d) Against a fresh table with the same index and buffer size, the digest and the allocated region are equal in all exits and every differing word lies in the free gap. The proven claim is therefore (a) and (b) with equal configuration; a deployment that must not leave old identifiers readable in the free gap has to clear it itself. Two consequences matter below. The table does not age: no sequence of scope entries and exits lengthens probe paths. And in the random-function model, for a schedule that does not depend on the hash, the law of the index at every moment is that of an insertion-only table with the current number of live names, so classical insertion-only results [8], [25] and the refusal law of Section VI apply to the next push at every moment up to the first refusal; Section VI states what holds afterwards.

Why is last-in-first-out order special? For a table built by $\mathrm{Ins}(x_1,\dots,x_n)$ let $c(x)$ be the cell of x and say that y *depends on* x, written $y \lhd x$, if $c(x)$ lies strictly before $c(y)$ on the probe path of y (Fig. 5). A deletion order is *clear-safe* if, after every prefix of it has been cleared, every remaining key is still found.

Theorem 2 (Safe orders): Fix the homes of $x_1,\dots,x_n$ ($2 \le n \le m$) under linear probing. (a) A deletion order is clear-safe for these homes if and only if it is a linear extension of $\lhd$, that is, y is deleted before x whenever $y \lhd x$. (b) It is clear-safe for every assignment of homes if and only if it is the reverse insertion order.

Proof: (a) Clearing never moves a key. The lookup of a remaining key y follows its probe path and stops at the first empty cell; the cells of that path before $c(y)$ were occupied, when y was inserted, by the keys x with $y \lhd x$. So y is found if and only if none of them has been cleared. (b) A key that depends on x is newer than x, so reverse insertion order is a linear extension of $\lhd$ for every assignment. Conversely, for $i < j$ give $x_1,\dots,x_{j-1}$ distinct homes and give x_j the home of x_i; then $x_j \lhd x_i$, so a universally safe order deletes x_j before x_i for every pair $i < j$. ∎

A scheme is *greedy* if a key is stored in the first empty cell of its own probe sequence and a search for a key follows the same sequence and stops at the first empty cell; part (a) holds for every greedy scheme with these lookups, and the converse in part (b) is proved here for linear probing only. The number of keys that depend on a key quantifies the cost of leaving

TABLE 3. Limit Law of the Number of Keys That Depend on a Uniformly Random Live Key, $\mathbb{P}(\mathrm{orph}=j)$, and Its Mean $\alpha/(2(1-\alpha))$.

α	j=0	1	2	3	4	mean
0.25	0.852	0.131	0.015	0.002	0.000	0.167
0.5	0.649	0.245	0.076	0.022	0.006	0.500
0.75	0.372	0.259	0.156	0.090	0.052	1.500
0.9	0.162	0.153	0.128	0.104	0.085	4.500

the discipline: it equals $Y_c - 1$, where Y_c follows the queue recursion $Y_c = (Y_{c-1}-1)^+ + A_c$ with A_c the number of keys homed at cell c, and for $n/m \to \alpha$ its law converges to that of a discrete M/D/1 queue, with mean and fraction of independent keys

$$\mathbb{E}[\mathrm{orph}] = \frac{\alpha}{2(1-\alpha)}, \qquad \mathbb{P}(\mathrm{orph}=0) \to \frac{(1-\alpha)(e^{\alpha}-1)}{\alpha}. \tag{2}$$

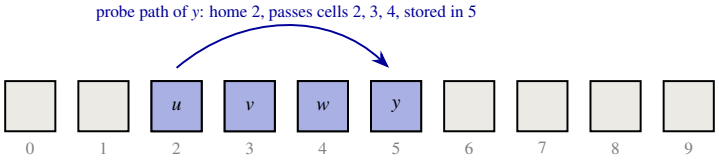
At load $3/4$ only 37 percent of keys could be removed out of order without harm, and clearing a random key orphans 1.5 keys on average (Table 3). The limit law is Janson's Robin Hood displacement law [25]; the reading as orphans of a deletion and the finite-size form are new as stated. This is why every deletion method for arbitrary order needs tombstones or shifting [8], [9], and why STRATA could not adopt a relocating deletion: its undo entries store index slots.

VI. BOUNDED MODE AND THE EXACT REFUSAL LAW

A. STATIC INDEX AND PROBE LIMIT

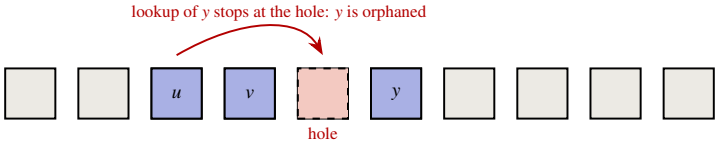
In the bounded mode the caller fixes the number of index slots m when the table is created. The index is never rebuilt; a declaration that would exceed the load limit is refused like one that exceeds the buffer. The caller may also set a probe limit L. A new name is inserted only if an empty slot lies among the first L slots of its probe path, and is refused with `probe-limit` otherwise (Algorithm 1). Since a stored name never moves and scope exit only empties slots, every stored name lies within L slots of its home, and a search may stop after L slots. One call thus examines at most L index words and, per fingerprint match, compares at most 64 name words (requirement R4). A name that hides a visible one needs no slot and is never refused for this reason.

From slot examinations to total work. A call reads its name once, at most 255 bytes, to pack and hash it (at most 51 packed or 64 raw words), examines at most L index words, and compares at most $\min(n_x, 64)$ name words for each of those whose fingerprint and meta word match, where n_x is the word count of the stored name. A declaration or lookup therefore performs at most $255 + L\,(1 + 64)$ elementary word operations whatever the names are; for $L = 258$ this is 17 025. The scratch space is one key of 268 bytes on the stack plus a handful of scalars, and no call is recursive. The bound excludes the scope exit, which is proportional to the number of declarations in the scope, the rebuild of the default mode, the generation of the key and the caller's own string handling, and, in growable mode, the allocator.



Dependency: $y \lhd x$ if the cell of x lies strictly before the cell of y on the probe path of y.
Here $y \lhd u$, $y \lhd v$, $y \lhd w$. A deletion order is safe iff every key is cleared before the keys it depends on (linear extension of $\lhd$).

Clearing w before y (not stack order):



Stack order is always safe: the newest key has no dependents, because a key's path crosses only cells that were filled before it. It is also the only order safe for every hash assignment (safe-orders theorem, part b). The number of keys orphaned by clearing one key is a queue length minus one (orphan law).

FIGURE 5. Dependency and orphans. Key *y* probed cells 2, 3 and 4 before it was stored in cell 5, so it depends on *u*, *v* and *w*. Clearing *w* first leaves a hole on its path and orphans *y*; clearing the newest key never does.

B. HOW OFTEN DOES THE LIMIT REFUSE AN HONEST NAME?

Probability model. Throughout this subsection the hash is an ideal random function: the homes of distinct names are independent and uniform on the m cells, and the schedule of operations does not depend on them. We consider an index of m cells holding $n = m - k$ live names, $k \geq 1$. By Theorem 1, the table without a probe limit is the insertion-only table of its live names at every moment, so the probability that the next fresh name needs more than L probes is that of an insertion-only linear-probing table with n keys. For $g \geq 1$ let $B(g) = g^{g-1}/g!$ and, for $j \geq 0$, $Q_j(s) = \frac{j}{s}\, s^{s-j}/(s-j)!$ for $s \geq j \geq 1$, with $Q_j(s) = 0$ for $s < j$ and $Q_0(s) = [s = 0]$. Four processes must be told apart, because the refusal rule makes the stored names depend on the hash.

(P1) *Ideal fill:* n fresh names are inserted into a table without a probe limit.
(P2) *First capped query:* the next fresh name is refused at limit L iff its probe count in (P1) exceeds L. This is what Theorem 3 computes, exactly.
(P3) *Conditioned fill:* the same query, given that none of the n earlier pushes would have been refused at limit L.
(P4) *Operational process:* a table with limit L that keeps receiving fresh names after refusing some of them; refused names are not stored.

The law is exact for (P2). It does not by itself describe (P3) or (P4), because the names that are accepted are selected by the refusal rule. Two facts give valid statements for them. For (P3), $\mathbb{P}(\text{refused} \wedge \text{clean fill}) \leq R_m(n, L)$, so the conditional probability is at most $R_m(n, L)/P_{\text{fill}}$, where $P_{\text{fill}} \geq 1 - \sum_{t<n} R_m(t, L)$ is the probability of a refusal-free fill (theory supplement [36]). For (P4) with insertions only, couple the operational table B with the ideal table A that receives the same names: the occupied cells of B are always contained in those of A, since a name that B refuses leaves a cell empty that A fills, and a name that B accepts lands on or before the cell where A places it, a cell that is already occupied in A if it lies before. A refusal by B needs L occupied cells from the home, hence the same in A, so the probability that the tth offered fresh name is refused is at most $R_m(t-1, L) \leq R_m(n_{\max}, L)$ whatever the earlier refusals, and the expected number of refusals among N offered fresh names is at most $N\, R_m(n_{\max}, L)$. The coupling argument covers insertions only. With scope exits, a name accepted by B and popped may free a cell that A keeps occupied by a different name, the inclusion can fail, and for a schedule that oscillates after refusals we claim only the bound up to the first refusal and the experiments below. The claims also require that the offered names are fixed independently of the hash (Section VI-C).

Theorem 3 (Exact refusal law): Let a ring of m cells hold $n = m - k$ keys, $k \geq 1$, with independent uniform homes under linear probing. The probability that a new key with an independent uniform home finds its first L cells occupied is

$$R_m(n, L) = \frac{k}{m} \sum_{g=L+1}^{n+1} (g - L)\, \pi_g, \qquad \pi_g = \frac{B(g)\, Q_{k-1}(m-g)}{Q_k(m)}. \tag{3}$$

TABLE 4. Smallest Probe Limit L With Per-Insertion Refusal Probability at Most ε: Exact Law for Index Sizes m, Its Limit, and the Limit the Chernoff-Type Bound Would Require.

load	ε	$m{=}2^8$	2^{10}	2^{12}	2^{14}	2^{16}	$m \to \infty$	Chernoff
3/8	10^{-6}	25	28	29	29	29	29	43
3/8	10^{-9}	37	44	46	47	47	47	62
1/2	10^{-6}	42	49	51	52	52	52	81
1/2	10^{-9}	59	77	83	84	85	85	117
5/8	10^{-6}	70	93	101	104	104	104	171
5/8	10^{-9}	95	142	162	168	170	170	244
3/4	10^{-6}	121	201	242	255	258	260	455
3/4	10^{-9}	148	289	380	413	422	426	638

Proof sketch: The sum ranges over block sizes $g \le n{+}1$. Cut the ring at its k empty cells. Each empty cell and the run of occupied cells after it form a block; by the cycle lemma [37] block sizes are Borel distributed, and, conditioned on filling the ring, the k blocks are exchangeable, so the first has size g with probability π_g by the Borel–Tanner identity [38]. A new key is refused exactly when its home starts a window of L occupied cells; a block of g cells contains $(g-L)^+$ such homes, and the expected number of blocks is k. The full proof is in the accompanying theory manuscript [36]. ■

$R_m(n, L)$ is nondecreasing in n (the occupied set of unrestricted linear probing does not depend on the insertion order and adding a key only adds a cell, so a window of L occupied cells persists), so the value at the load limit covers every lower load. Boundary cases are handled explicitly: for $k = 1$ the law reduces to $(m-L)^+/m$. It is evaluated in logarithms with the log-gamma function, because g^g and s^s overflow double precision at $m = 65\,536$, and it was compared with brute-force enumeration of every home sequence in exact rational arithmetic and with an independent derivation (Section VII). As $n/m \to \alpha$ it converges to $(1-\alpha)\sum_{g>L}(g-L)\,\mathrm{Borel}(g;\alpha)$, and with $\rho = \alpha e^{1-\alpha}$

$$R_\infty(\alpha, L) \sim \frac{(1-\alpha)\,\rho^{L+1}}{\alpha\sqrt{2\pi}\,(1-\rho)^2\,L^{3/2}} \qquad (L \to \infty). \tag{4}$$

The quantity is classical: it is the tail of the unsuccessful-search cost, whose generating function and Poisson limit are given by Janson and Viola [24] and whose finite-size values follow from Knuth's run-length counting [8]. What is new here is the short derivation of the explicit sum (3), its validation against the implementation, and its use as a design rule. A Chernoff bound on the number of homes in an interval, which an earlier version of this study used, gives $\rho^L/(1-\rho)$; by (4) it is loose by a factor growing like $L^{3/2}$. At load $3/4$ and 65 536 slots the bound exceeds the exact value by a factor of 315 at $L = 64$ and 4 637 at $L = 512$.

Table 4 turns the law into design values by bisection on L, which is valid because $R_m(n, L)$ is nonincreasing in L. At load $3/4$ and $\varepsilon = 10^{-6}$ the exact limit is 258 for $m = 2^{16}$ slots (the figure quoted in this article always refers to that index size; it is 260 as $m \to \infty$), against 455 for the Chernoff-type rule, 43 percent fewer probes allowed per call; it is smaller for small indexes (121 at 256 slots).

Per request and per compilation. The value ε is a probability per fresh-name request, not per compilation. A compilation that offers N fresh names has, for an oblivious schedule, a probability of at least one refusal of at most $N\,\varepsilon$ by the union bound (exactly, for a monotone fill, the theory supplement gives $1 - P_{\mathrm{fill}}$, which lowers the limit for a whole-fill target of 10^{-6} at $m = 4096$ from 402 to 306 probes), and the expected number of refused names is at most $N\,\varepsilon$ by the coupling above. For instance, $N = 10^4$ fresh names at $\varepsilon = 10^{-6}$ give at most 10^{-2} for the compilation; a target of 10^{-6} for the whole compilation needs $\varepsilon = 10^{-10}$. Repeated names do not consume slots, but a name that is popped and pushed again counts as a new request, and these requests are correlated; a retry of a refused name with an unchanged key and table is refused again, so retries add no independent chances. A schedule that adapts to observed refusals depends on the hash, and only the unconditional bound of L probes holds for it. The theory supplement also gives the probability that a persistent set of names can never refuse a fresh one and a schedule-free bound for bounded churn above persistent names [36].

C. KEYED HASH AND THE LIFETIME OF A KEY

The default hash is the 32-bit MurmurHash3 mixer [39] applied to the name words. As an option, the table hashes with HalfSipHash with one compression and three finalization rounds [5], [32] under a 64-bit key supplied by the caller, checked against the reference code [40]. Theorem 3 is a statement in the ideal model above. A keyed 32-bit hash with a 64-bit key is not a random function, and fresh keys do not make the homes of all names exactly independent; using the law for the practical table relies on the assumption that HalfSipHash-1-3 under an unknown key is indistinguishable from a random function by the adversary, which is its design goal and not a theorem, and the empirical agreement of Section IX-I is evidence for names that are not adversarial. We make no cryptographic claim.

Under the assumption above, Theorem 3 applies to chosen names only if the key is independent of everything the adversary has seen. If every table is created with a fresh key and the adversary is oblivious, that is, fixes its names before the key is drawn, the homes are modeled as independent and uniform and the law is used for every name set and scope schedule. A fresh key per translation unit limits what can be learned across tables; it does not by itself address an adaptive adversary who queries the same table, whose refusals or probe lengths reveal information about the key. If one key is reused across tables and the adversary can observe probe lengths, it can learn names that collide with a target cell and plant a long run of occupied cells: in an experiment against the real table with an idealized probe-length oracle ($m = 2\,048$, $L = 512$, 700 honest names, five seeds), 1 to 5.5 percent of honest names on average (6.1 percent at most over seeds) were refused with a persistent key and none with a fresh key or with the same number of random names (Table 5). The rule is therefore one key per translation unit, derived from a device secret and a never-repeating counter, which must survive a restart

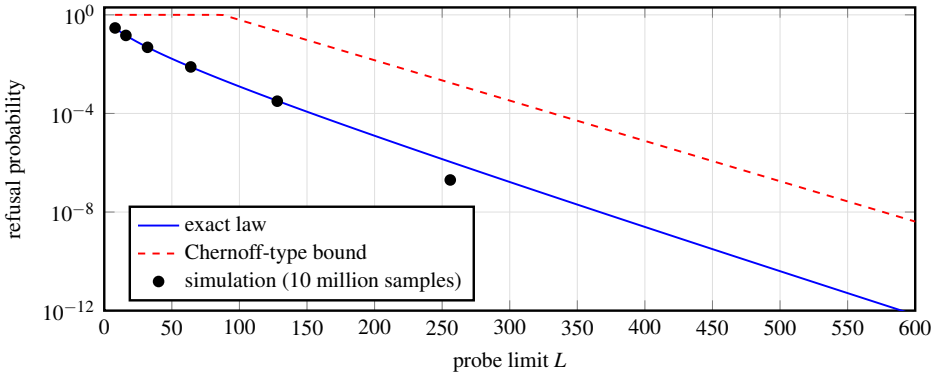


FIGURE 6. Probability that an insertion needs more than L probes ($m = 65\,536$, load $3/4$): exact law (Theorem 3), the Chernoff-type bound, and a stored simulation of 10 million samples from 500 tables.

TABLE 5. Honest Names Refused by the Probe Limit (Percent of 700) After the Adversary's Learned Colliders Are Declared: Persistent Key (Mean and Maximum Over Five Seeds), Fresh Key, and Random Names.

colliders	persistent key (mean)	(max)	fresh key	random names
400	1.00%	1.29%	0.00%	0.00%
512	5.51%	6.14%	0.00%	0.00%
600	4.71%	6.00%	0.00%	0.00%
700	5.14%	5.71%	0.00%	0.00%
800	5.14%	6.00%	0.00%	0.00%

of the device (a counter that restarts at zero repeats keys). The library only takes a caller-supplied 64-bit key; neither key derivation nor counter persistence is implemented, and our experiments use pseudo-random keys (flooding, refusal-law and adaptive runs) or one constant key (the keyed timing variant, the bounded traces and the embedded image), so the timing of the keyed variant says nothing about key derivation cost. The work bound of R4 does not depend on the key: a call examines at most L slots whatever the names. The experiment above uses an idealized probe-length oracle; it measures what an adversary could learn if such a channel existed, not a timing channel.

D. COLLIDING IDENTIFIERS FOR THE DEFAULT HASH

The mixer processes a name word k as $h \leftarrow \mathrm{rotl}_{13}\big(h \oplus f(k)\big) \cdot 5 + c$, where f multiplies, rotates and multiplies and is a bijection on 32-bit words that does not involve the seed. Following Aumasson and Bernstein [5], [31], flipping bit 18 of $f(k_1)$ flips bit 31 of the rotated word; multiplying by 5 and adding c leaves that difference in bit 31 for every h; and flipping bit 31 of $f(k_2)$ in the next word cancels it. Two consecutive words can thus be exchanged without changing the hash, whatever the seed. Here a word must be a packing of five identifier characters, and a random search finds word pairs that satisfy the constraints for both alternatives within a fraction of a second. Concatenating t such pairs gives 2^t distinct identifiers of $10t$ characters with one common hash; with $t = 15$ this yields 32 768 valid C identifiers of 150 characters, which collide under the built-in seed and eight further seeds.

VII. VERIFICATION

Unit and randomized differential tests. A test program compares the table with an independent reference model (a separate implementation of the semantics in terms of standard containers that shares no code with the table) after every operation, in configurations that cover packed and raw names, fixed and growable buffers, a dense and a sparse index, failing allocators, and a hook that forces every key into one probe cluster. It requires a 64-bit digest of the complete state to be unchanged after every refusal and rebuilds a fresh table from the live declarations to compare memory. It makes 2 732 358 checks under AddressSanitizer and UndefinedBehaviorSanitizer with two compilers.

Tests added after the review of this article. (i) *Direct comparison on refusal:* 783 955 refusals were checked by comparing a saved byte-for-byte copy of the whole buffer, the options, the counters and the digest, not the digest alone; all were unchanged. The sweeps cover the capacity word by word, an exact threshold test (a call is accepted iff the free words reach the need: 529 probes at exactly the need accepted, 1 592 at one word less refused), growable budgets and failure of the allocator at each of its first twelve calls. (ii) *Status precedence* was printed and checked: an invalid table gives `no-memory`; `empty-name` and `name-too-long` precede any memory check; `too-deep` precedes `no-memory` on entry; `no-memory` precedes `probe-limit`; `no-scope` on exit at depth zero. (iii) *Encodings:* round trips of packed and raw names of every length from 1 to 255, names that differ only in padding or in embedded zero bytes, high-bit bytes, all 256 type tags and depth 16 383, with 1 030 constructed names pairwise distinct in both modes. (iv) *State equality* at the four levels of Section V. These tests found no defect, and the library was not changed.

Bounded exhaustive enumeration. A second program starts every sequence from a freshly created table, enumerates every sequence of nine operations over {enter, exit, declare a, b, c, d} ($6^9 = 10\,077\,696$ per configuration) in eight small config-

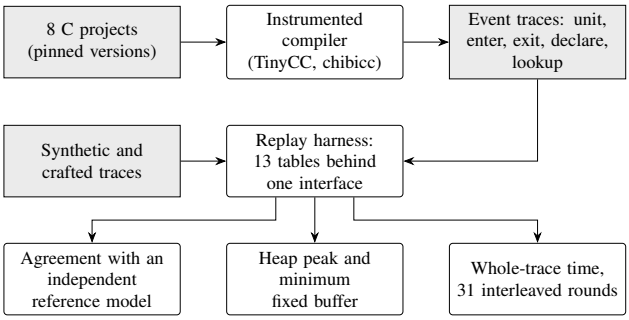


FIGURE 7. Evaluation method. Symbol-table events are logged by instrumented compilers and replayed on every table; an independent reference model decides correctness.

urations (fixed buffers of 128 to 256 bytes or a static index of 16 slots, packed and raw names, a forced single probe cluster, the keyed hash, probe limits 0, 1 and 2) chosen so that every kind of refusal occurs, over the alphabet of the two scope operations and four declarations of different names (including a non-identifier name stored raw). The buffer is refilled with a marker pattern and the reference model is recreated for every sequence. After each operation the result and the lookups of all four names are compared with the model and the invariants are checked, and after each refusal the digest is compared (not the bytes). It counts each executed operation: 80 621 568 sequences, 725 594 112 operations, 130 361 632 of them refused (a refusal being a returned error status other than exit at the outermost scope), no failure. It was run again on the final code (3 min 21 s). It is a finite check and not a proof of correctness.

Differential fuzzing and mutation. A libFuzzer harness ran 9.8 million executions on the code before the change of Section IX-F and 2.5 million on the final code without a failure, and detected each of 9 injected single-line defects within a second; a further 60-second run on the final code (348 786 executions) found nothing. The tests listed here were rerun on the final code after the last change to the library.

The analysis. Theorem 1 was checked in 488 919 state comparisons over five insertion rules, Theorem 2 on every assignment and every deletion order of small rings (397 562 pairs), and the refusal law against brute-force enumeration of every home sequence in exact rational arithmetic (258 cases) and against an independent derivation by Knuth's Abel-binomial route (481 cases). The proofs are pen-and-paper; no proof assistant was used.

VIII. EXPERIMENTAL SETUP

A. WORKLOADS

The workloads are sequences of symbol-table events produced by a real compiler (Fig. 7). We added logging to the Tiny C Compiler [7] where it looks up an identifier, pushes a symbol, and opens or closes a scope, and compiled eight open-source C projects at pinned versions (Table 6). At the start of each translation unit every table is destroyed and created anew, as a compiler would do. *Trace semantics.* A

TABLE 6. The Eight Compiler Traces. "Names" Counts Distinct Identifiers; "Peak" Is the Largest Number of Simultaneously Live Declarations.

Project	Units	Events	Declares	Lookups	Names	Peak	Depth
cJSON	2	20 874	5 247	9 194	1 366	1 067	10
littlefs	2	21 080	3 835	12 230	1 001	732	16
zlib	15	58 058	16 341	27 405	1 041	571	23
Lua	34	188 509	58 619	86 887	3 622	1 365	14
FreeRTOS	9	37 875	10 346	17 417	2 226	1 505	16
Mbed TLS	105	983 008	319 886	438 876	7 622	2 302	14
TinyCC	1	89 135	13 776	54 623	6 704	5 509	27
SQLite	1	268 960	27 072	172 426	7 801	4 245	22

trace is a text file with one event per line: a unit start, scope entry and exit, a declaration and a lookup with an identifier. The API has one namespace, and the traces log only the ordinary-identifier namespace of C (variables, functions, typedef names, enumeration constants). Struct, union and enum tags, members, labels and macros are separate tables of the compilers and are not logged. In TinyCC the hooks sit in `sym_find` (every lookup, including those that find nothing, which are 12.5 to 34 percent of the lookups), in `sym_link` for non-tag symbols, and in the block, prototype and function-parameter scopes; file-scope pushes made while a local scope is open (implicit declarations) are logged separately and ignored by every replay. A declaration of a name already declared in the same scope is replayed as the API's same-scope replacement; the eight TinyCC traces contain none, while the chibicc traces contain 1 to 122 per project. Empty names that occur in chibicc (9 324 lines) are dropped by the replay tools and stored as a single placeholder byte in the compiler integration. Sources are pinned by version tag or URL only; commit hashes and archive checksums were not recorded, and SHA-256 checksums of every trace file are published with the artifact. Three Mbed TLS units that TinyCC could not compile are skipped. The reference model that decides correctness is a separate implementation of the semantics above and shares no code with the table.

A second set of seven traces from an instrumented chibicc [41] is used for the bounded-mode experiment, and nine synthetic traces stress cases that C programs rarely produce.

B. TABLES COMPARED

13 tables are driven through one interface.

- STRATA and four variants: raw instead of packed names, a sparse index (at most half full), no shrinking of the buffer, and the keyed hash.
- *The conventional arena table*, our main baseline: bucket heads, a stack of entries (chain link, hash, value, length and depth, unpacked name) and scope marks in one buffer, with the same storage discipline as STRATA. It was written as the strongest conventional design we could build, so that a comparison isolates the compact layout from the mere absence of an allocator.
- One standard-library hash map per scope; an undo log over a standard-library map (the algorithm of LLVM's

`ScopedHashTable`); the same over a Swiss-table map [42]; an interning Swiss-table map.
- METIS-X [17], the predecessor, a Robin Hood table with one slot per declaration, and two further tables from its repository, used unmodified.

C. METRICS AND PROCEDURE

Correctness is agreement with the reference model on every lookup and declaration result. *Heap peak* counts every allocation with its usable size. The *smallest fixed buffer* is found by bisection, at a resolution of 64 bytes, as the smallest buffer with which the whole trace replays without a refusal. *Replay time* is the time to replay a whole trace on a fresh table. In each of 31 measured rounds, after two warm-up rounds, every table replays the trace once and the starting table rotates. For two tables we report the median over rounds of the ratio of their times (pairing is by round: both tables are replayed in the same round) with a 95 percent bootstrap interval from 2000 resamples of rounds of the percent change $100\,(r-1)$, and call a difference *established* when the interval excludes zero ("n.e." otherwise); an established difference is not an equivalence claim, and a non-established one is not evidence of equal speed. Section IX-E repeats the comparison in independent sessions with an equivalence test.

D. ENVIRONMENT

All host measurements were taken on one machine: Intel Core Ultra 7 258V, Windows Subsystem for Linux 2 with Ubuntu 22.04 (Linux 6.18.33.2-microsoft-standard-WSL2), g++ 11.4.0 with `-O2`, the benchmark pinned to one core by `taskset` (the chibicc end-to-end driver is not pinned), run `20261013-143828-paper` of the public artifact [36]. The repeated sessions of Section IX-E ran on a different, shared native Linux machine described there. The embedded experiment uses `arm-none-eabi-g++` 10.3.1 with `-Os` and QEMU [43] 6.2 emulating a Cortex-M3 board (`mps2-an385`) restricted to 64 KiB of memory, without heap and without operating system. The refusal law was checked against the real bounded-mode table, with the keyed hash and a fresh random key per trial, in seven settings of 400 000 trials.

IX. RESULTS

A. CORRECTNESS

All 13 tables agree with the reference model on the eight Tiny C Compiler traces and on the seven chibicc traces: 195 table and trace pairs, no wrong lookup, no wrong declaration result, no refused declaration. On the synthetic traces METIS-X refuses 149 of 400 declarations when one name is shadowed in 400 nested scopes.

B. MEMORY

STRATA replays the traces in fixed buffers of 15.8 to 156.5 KiB, which is 27 to 36 bytes per live declaration at the peak (Fig. 8, Table 7). The conventional arena table needs a buffer that is larger on every trace, so that STRATA saves

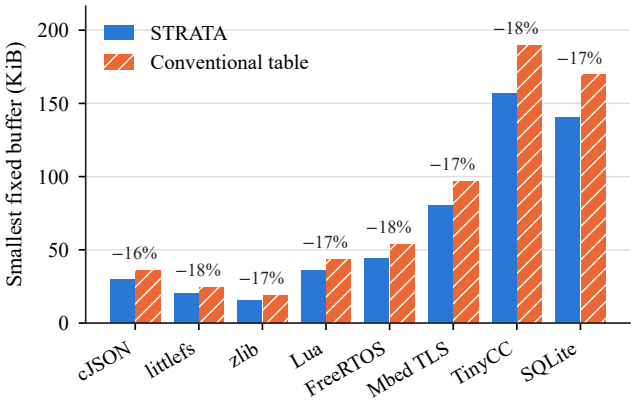


FIGURE 8. Smallest fixed buffer with which each compiler trace replays without a refusal. The label gives the saving of STRATA over the conventional arena table.

16.4 to 17.9 percent; Section IX-C explains where this saving comes from.

Against tables that allocate, the difference is larger and has a different cause. The heap peak of STRATA in growable mode is 23 to 76 percent (median 58 percent) below that of METIS-X, and STRATA makes 98.7 to 99.9 percent fewer allocator calls. In growable mode the conventional arena table reaches the same heap peak as STRATA on 6 of the 8 traces, because both grow one block in steps. Most of the gain over the predecessor is therefore due to keeping everything in one allocation-free buffer, not to the compact layout.

C. ANATOMY OF THE SAVING: AN EXACT SPACE LEDGER

Every single-buffer layout considered here uses a number of words that is a sum of per-element costs, as in (1). Let C_0 be the conventional layout, C_1 the same without the stored hash word, C_2 an open-addressed index over one entry per declaration (a layout rule evaluated by accounting only, no code), S_{raw} STRATA's layout with raw names and S STRATA itself. Only C_1 was a layout rule at first; it is now implemented and measured (below). Subtracting the formulas gives exact identities at every instant:

$$U_{C_0} - U_{C_1} = d, \qquad U_{S_{\mathrm{raw}}} - U_S = \sum_{x \text{ packable}} \left(\left\lceil \frac{\ell_x}{4} \right\rceil - \left\lceil \frac{\ell_x}{5} \right\rceil \right), \tag{5}$$

and interning with an undo log saves the names of hidden declarations but costs one extra word per open scope. The accounting reproduces all 24 measured minimum buffers to the 64-byte resolution of the measurement. On the eight traces, dropping the stored hash saves 9.2 to 12.2 percent and packing a further 5.3 to 9.0 percent, while interning changes the footprint by between -0.38 and 0.00 percent (Fig. 9, Table 9).

The stronger baseline, measured. We implemented C_1 as a real table: the conventional arena table with the same hash, load factor, growth and refusal behavior, entries of $3 + w$ words, and the hash recomputed from the stored name wherever the stored word was used. Its measured smallest buffer

TABLE 7. Memory on the Eight Compiler Traces. Smallest Fixed Buffer of the Two Single-Buffer Tables, and Heap Peak in Growable Mode of Six Tables (KiB).

	Smallest fixed buffer				Heap peak, growable mode (KiB)					
Project	STRATA (KiB)	Arena (KiB)	Saving (%)	Bytes per live decl.	STRATA	Arena	METIS-X	Map per scope	Undo log	Interning map
cJSON	30.0	35.9	16.4	28.8	42.9	42.8	102.7	72.0	116.0	140.4
littlefs	20.0	24.2	17.5	28.0	25.8	25.7	106.9	54.1	67.8	141.9
zlib	15.8	18.9	16.6	28.2	17.2	25.7	52.6	44.4	60.3	71.1
Lua	35.9	43.4	17.3	27.0	64.2	96.2	106.8	96.7	127.7	142.3
FreeRTOS	44.0	53.6	17.9	29.9	57.8	57.7	217.3	116.1	147.3	293.1
Mbed TLS	80.0	96.9	17.4	35.6	144.3	144.2	234.4	189.4	277.7	310.9
TinyCC	156.5	189.8	17.5	29.1	324.5	324.4	423.0	424.5	550.8	575.6
SQLite	140.4	169.4	17.2	33.9	194.8	194.7	451.0	333.3	513.5	1125.6

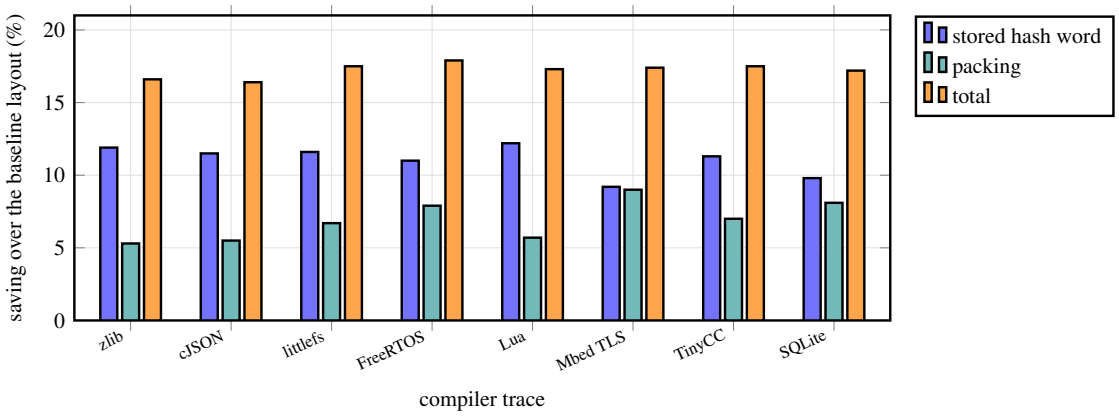


FIGURE 9. Saving of STRATA's layout over the conventional single-buffer layout on the eight traces, split into the stored hash word and name packing, from the exact space ledger.

TABLE 8. Smallest Fixed Buffer (KiB, 64-Byte Resolution) of STRATA, the Conventional Arena Table With (C_0) and Without (C_1) the Stored Hash Word and STRATA With Raw Names, and the Saving of STRATA Over C_0 and C_1 (Percent).

Project	STRATA	C_0	C_1	STRATA raw	vs. C_0	vs. C_1
zlib	15.8	18.9	16.6	16.7	16.6	5.3
cJSON	30.0	35.9	31.8	31.8	16.4	5.5
littlefs	20.0	24.2	21.4	21.5	17.5	6.7
FreeRTOS	44.0	53.6	47.8	47.8	17.9	7.9
Lua	35.9	43.4	38.1	38.1	17.3	5.7
Mbed TLS	80.0	96.9	87.9	87.9	17.4	9.0
TinyCC	156.5	189.8	168.2	168.3	17.5	7.0
SQLite	140.4	169.4	152.8	152.9	17.2	8.1

equals the ledger's prediction exactly on all eight traces, and at that buffer all four tables agree with the reference model with no refusal, while 64 bytes less makes each refuse. Against C_1, STRATA needs 5.3 to 9.0 percent less (7.5 percent pooled), against C_0 16.4 to 17.9 percent (17.3 percent pooled; Table 8); STRATA with raw names needs the same buffer as C_1 to within 64 bytes, so the remaining saving is name packing. The headline saving of one sixth is therefore a comparison with a conventional table that stores a hash word, and the saving against the stronger baseline is about one thirteenth. The smaller baseline costs time: C_1 replays 4 to 12 percent slower than C_0 (Section IX-E). Interning pays only in programs that shadow often, as the sweep of Section IX-J confirms.

What is counted. The buffer is the single caller-supplied block: for STRATA the index, the identifier records and the undo and scope stacks; for the arena tables the bucket heads, the entry stack and the scope marks. Not counted are the control structure (144 bytes for STRATA, 48 for each arena table), the stack key of one operation (268 and 256 bytes), a 256-byte static code table in STRATA, and the compiler's own state; in fixed mode no heap is used. Including them adds a constant of about 100 bytes in favor of the arena tables. The saving is that of the symbol table alone, not of a compiler. The bisection assumes that success is monotone in the capacity; we tested this on all eight traces and the four fixed-buffer tables (about 120 000 capacities: every 64 bytes up to twice the minimum, every 4 bytes within ± 2 KiB of it, every 4 KiB up to eight times): no non-monotone point and no wrong result, and the threshold is sharp, at exactly the peak number of words in use. This holds for fixed mode on these traces; the STRATA index size is fixed by the buffer there. The measurement resolution of 64 bytes is under 0.5 percent of the smallest buffer. The peak is taken over live declarations, so the denominator of bytes per declaration is the peak of simultaneously live declarations, not of distinct names.

D. REPLAY TIME

Table 10 gives the paired comparisons of the first run. Against the conventional arena table, STRATA is faster on 0 traces, slower on 2, and on 6 the difference is not established; the

TABLE 9. Smallest Fixed Buffer (KiB) of Five Layouts and the Savings (Percent) of Dropping the Stored Hash ($C_0 \to C_1$), of Packing ($C_1 \to S$) and in Total ($C_0 \to S$).

trace	C_0	C_1	C_2	S_{raw}	S	hash	pack	total
zlib	18.9	16.6	16.6	16.7	15.8	11.9	5.3	16.6
cJSON	35.9	31.8	31.8	31.8	30.0	11.5	5.5	16.4
littlefs	24.2	21.4	21.4	21.5	20.0	11.6	6.7	17.5
FreeRTOS	53.6	47.8	47.8	47.8	44.0	11.0	7.9	17.9
Lua	43.4	38.1	38.1	38.1	35.9	12.2	5.7	17.3
Mbed TLS	96.9	87.9	87.9	87.9	80.0	9.2	9.0	17.4
TinyCC	189.8	168.2	168.2	168.3	156.5	11.3	7.0	17.5
SQLite	169.4	152.8	152.8	152.9	140.4	9.8	8.1	17.2

point estimates lie between -10.6 and $+10.7$ percent. The compact table is not faster than the conventional one, and where a difference is established it is at most 10.7 percent. Against METIS-X, STRATA is faster on 8 of 8 traces, by 15.9 to 36.7 percent, and the conventional arena table is faster than METIS-X on 8 of 8, by 14.6 to 34.8 percent. Per operation (Table 11), the 95th-percentile latency of a declaration is 56 to 66 percent lower with STRATA than with METIS-X and that of a scope exit 71 to 82 percent lower; lookup latency differs by between -20.3 and $+4.9$ percent without a consistent direction.

E. REPEATED SESSIONS ON A SECOND CONFIGURATION

Because the verdicts of single runs changed between runs, we repeated the replay-time comparison in five independent sessions: a new process per trace, 31 paired rounds after two warm-up rounds as before, the start order rotated by session, the process pinned to one core, sessions five minutes apart, with the stronger baseline C_1 added. The sessions ran on a different configuration from the first run: a native Linux desktop (kernel 7.2.4, `g++` 16.2.1, Intel Core i7-13620H, `intel_pstate` governor `powersave`, turbo enabled), not WSL2 with `g++` 11.4. The machine is shared and not isolated: the frequency of the pinned core at the start of a session ranged from 0.6 to 4.7 GHz, its sibling hardware thread was 1.4 to 5.8 percent busy, and the load average at the start was 0.4 to 2.5; absolute times differed by up to a factor of 1.5 between sessions, so only ratios are interpreted. The unit of replication is the session. For each pair of tables and trace we give the pooled median ratio of paired times with a 95 percent t interval over the five session medians (4 degrees of freedom). "Slower" or "faster" means that the interval excludes 1. To test equivalence we use two one-sided tests with a stated practical margin, a ratio in $[0.90, 1.10]$: "eq" if the 90 percent interval lies inside the margin, "out" if it lies outside, "inc" otherwise. The margin is a choice, not derived from the data. The comparisons are exploratory, not confirmatory; 32 intervals are reported without correction for multiplicity, and the eight traces are correlated through shared compiler behavior, so they are not eight independent samples of programs.

The result differs from the first run (Table 12). Against the conventional arena table STRATA is slower on 8 of 8 traces, by 8 to 20 percent, and for six of them the equivalence margin of ± 10 percent is excluded. Against the stronger baseline C_1 it is slower on 6 of 8 traces by 0 to 13 percent, and equivalent within the margin on 6. The conventional table without the hash word is itself slower than with it on 7 of 8 traces, by 4 to 12 percent. Against METIS-X, STRATA is faster on 8 of 8 traces, by 7 to 31 percent; the keyed hash costs 20 to 28 percent. The sign of the difference to the arena table therefore did not reverse, but the earlier statement that no difference was established on most traces holds for the first run only; the first run and these sessions differ in compiler, kernel, processor and scheduling, and we cannot separate these factors. The practical conclusion is that the compact table is not faster than a conventional single-buffer table, and that in these sessions it is slower by 8 to 20 percent against the table with a stored hash and by up to 13 percent against the one without.

Measurement details. A replay time is one `std::chrono::steac` interval (`clock_gettime` with `CLOCK_MONOTONIC`, 1 ns resolution) around the replay of a whole trace, with no batching; table creation for each translation unit is inside the interval. The empty interval costs a median of 20 ns (18 ns for raw `clock_gettime`, 13 ns for `rdtscp`), negligible against replay times of hundreds of microseconds or more. Per-operation latencies (13 to 79 ns) are taken with `rdtscp` with the median empty-pair overhead subtracted, which adds a fixed cost to every operation and may change the measured cost of the instrumented operation; the 95th percentile is taken over all samples of a whole trace pooled across translation units, as the median of three replays, so a translation unit with many operations weighs more. Raw paired rounds, the per-session context (load, frequency, busy fractions) and the exact commands are published with the artifact.

Instruction counts. Wall-clock time on a shared machine depends on clock frequency and load, and the microcontroller runs under an emulator that provides no timing. We therefore also counted the user-space instructions that a replay executes, with the processor's performance counters (`benchmarks/instr_count.cpp`). Such a count does not depend on frequency or on other programs: the five replays of a table on a trace differed by at most 0.03 percent (Table 13). STRATA executes 35 to 48 percent more instructions than C_0 and 30 to 40 percent more than C_1 on every trace, and the keyed variant 44 to 53 percent more than unkeyed STRATA. We have not profiled where the extra instructions go; the packed name, the fingerprint check and the undo stack are the likely sources. The count agrees in sign with the repeated sessions, which found STRATA slower, but not in size: the extra instructions cost 8 to 20 percent of time, because the replay is partly bound by memory accesses that are common to all tables. An instruction count is a property of the compiled code, not of a machine, so it is the figure that carries over to a processor we did not measure; it is not a time, and we do not convert it into one. Because the count is reproducible to within 0.03 percent, it is not subject to the instability that affected the timing verdicts.

TABLE 10. Replay Time on the Eight Compiler Traces: Median Ratio Over 31 Paired Rounds, in Percent, With 95 Percent Bootstrap Interval. Negative Means That the First Table Is Faster; "n.e." Means Not Established.

Project	STRATA vs. arena table		STRATA vs. METIS-X		Arena table vs. METIS-X		Keyed vs. default hash
cJSON	+0.2 [−3.9, +1.6]	n.e.	−22.8 [−27.5, −19.1]	faster	−21.3 [−23.8, −19.8]	faster	+20.2 [+18.5, +24.6]
littlefs	+7.2 [+2.4, +11.0]	slower	−24.5 [−27.2, −18.1]	faster	−26.5 [−28.9, −22.7]	faster	+24.6 [+18.5, +29.8]
zlib	−10.6 [−15.6, +4.1]	n.e.	−29.9 [−32.8, −27.4]	faster	−20.4 [−26.9, −15.6]	faster	+32.2 [+16.1, +41.4]
Lua	−2.7 [−13.3, +3.6]	n.e.	−30.9 [−35.3, −29.2]	faster	−28.4 [−33.8, −25.2]	faster	+22.7 [+19.1, +35.2]
FreeRTOS	+0.4 [−4.0, +6.6]	n.e.	−33.9 [−39.0, −31.3]	faster	−34.8 [−36.5, −32.9]	faster	+20.3 [+15.2, +22.4]
Mbed TLS	+0.2 [−2.8, +3.3]	n.e.	−36.7 [−39.6, −32.4]	faster	−34.8 [−38.3, −32.2]	faster	+27.1 [+23.0, +34.3]
TinyCC	+10.7 [+5.9, +12.7]	slower	−20.4 [−25.7, −18.7]	faster	−27.5 [−33.7, −22.3]	faster	+20.6 [+13.1, +22.0]
SQLite	−1.1 [−3.7, +2.1]	n.e.	−15.9 [−20.7, −6.3]	faster	−14.6 [−19.5, −10.0]	faster	+22.3 [+18.7, +33.8]

TABLE 11. 95th-Percentile Latency per Operation in Nanoseconds for STRATA (S), the Conventional Arena Table (A) and METIS-X (M).

	Lookup			Declare			Scope exit		
Project	S	A	M	S	A	M	S	A	M
cJSON	49	41	49	62	71	141	13	19	56
littlefs	51	41	49	65	72	187	19	26	90
zlib	44	41	44	56	68	136	16	23	59
Lua	53	89	54	66	130	164	21	56	84
FreeRTOS	52	44	54	68	73	180	15	23	87
Mbed TLS	66	54	69	79	78	202	27	36	109
TinyCC	51	44	51	66	75	188	20	24	76
SQLite	52	53	65	71	86	204	22	28	76

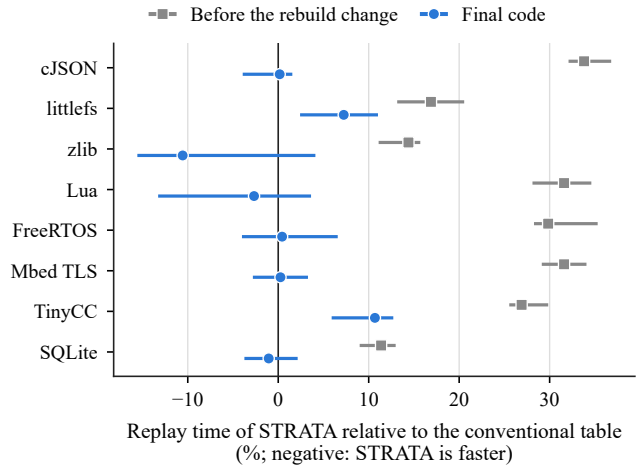


FIGURE 10. Replay time of STRATA relative to the conventional arena table before and after the rebuild change of Section IV-E: median of paired ratios with 95 percent bootstrap interval.

F. AN EARLIER TIME PENALTY WAS AN IMPLEMENTATION ARTIFACT

An earlier version of STRATA was slower than the conventional arena table on every trace, by 11.4 to 33.8 percent (Fig. 10). Per-operation latencies showed that lookups were level and the whole difference was in declarations, caused by redundant rebuild work (rebuilding on every buffer growth, hashing each name twice per rebuild, and a second probe walk for a new name). The changes of Section IV-E remove it and alter no result other than time. We keep the episode because the first measurement supported a plausible but wrong conclusion.

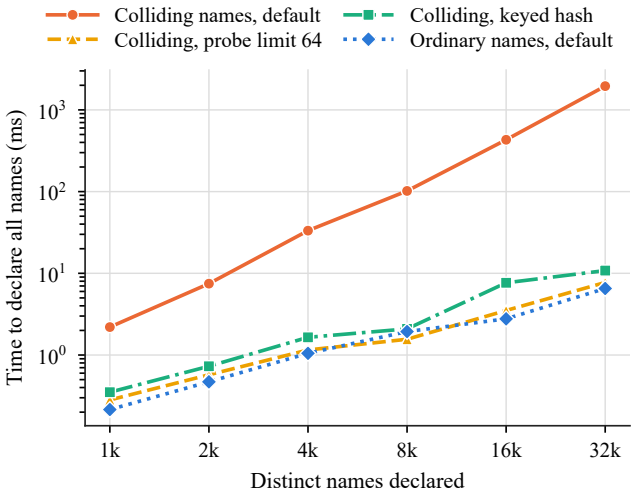


FIGURE 11. Time to declare *N* distinct names of 150 characters. Colliding names are those of Section VI-D; ordinary names are random identifiers of the same length. Both axes are logarithmic.

G. ABLATION AND SYNTHETIC TRACES

Table 14 isolates three design choices. Storing names as raw bytes instead of packing them needs a 5.5 to 9.0 percent larger buffer and is slower on 2 of 8 traces and faster on 0, because a packed name has fewer words to hash and compare. Keeping the index at most half full makes replay faster on 6 of 8 traces (changes between −4.5 and −12.9 percent) and needs a 18 to 27 percent larger buffer, which more than cancels the memory advantage: index load is the lever that trades the saving for speed. On the 9 synthetic traces (Table 15) STRATA is faster than the conventional table on 6 and slower on 1: faster where names are hidden often, where scopes are entered and left at a high rate, and with short names, and slower on the trace dominated by lookups.

H. HASH FLOODING AND ITS DEFENSES

We declare and then look up the 32 768 colliding identifiers and as many random identifiers of the same length (Fig. 11, Table 16). In the default configuration the colliding names take 1954 ms to declare against 6.5 ms for ordinary names, a factor of 300, and 1684 ms to look up; time grows with exponent 1.96 in the number of names, since every new name scans the cluster formed by all earlier ones. With the keyed hash the factor is 1.2: names built against the unkeyed mixer

TABLE 12. Replay Time in Five Independent Sessions: Pooled Median Ratio of Paired Times With 95 Percent t Interval Over Sessions, Verdict (Interval Excludes 1 or "n.e.") and Equivalence Test Against ±10 Percent (eq, out, inc). Ratio Below 1 Means That the First Table Is Faster.

	STRATA / C_0			STRATA / C_1			C_1 / C_0			STRATA / METIS-X		
Project	Ratio [95% CI]	Verdict	Eq.	Ratio [95% CI]	Verdict	Eq.	Ratio [95% CI]	Verdict	Eq.	Ratio [95% CI]	Verdict	Eq.
zlib	1.110 [1.095, 1.131]	slower	inc	0.995 [0.976, 1.007]	n.e.	eq	1.120 [1.102, 1.139]	slower	out	0.769 [0.764, 0.773]	faster	out
cJSON	1.158 [1.143, 1.174]	slower	out	1.043 [1.038, 1.048]	slower	eq	1.111 [1.099, 1.123]	slower	out	0.828 [0.823, 0.831]	faster	out
littlefs	1.204 [1.192, 1.215]	slower	out	1.108 [1.090, 1.122]	slower	inc	1.084 [1.061, 1.113]	slower	inc	0.825 [0.812, 0.836]	faster	out
FreeRTOS	1.192 [1.173, 1.204]	slower	out	1.072 [1.059, 1.085]	slower	eq	1.110 [1.086, 1.128]	slower	inc	0.708 [0.705, 0.712]	faster	out
Lua	1.176 [1.156, 1.193]	slower	out	1.068 [1.052, 1.080]	slower	eq	1.102 [1.095, 1.107]	slower	inc	0.757 [0.755, 0.762]	faster	out
Mbed TLS	1.166 [1.161, 1.170]	slower	out	1.047 [1.037, 1.055]	slower	eq	1.112 [1.105, 1.119]	slower	out	0.692 [0.689, 0.693]	faster	out
TinyCC	1.191 [1.154, 1.226]	slower	out	1.131 [1.114, 1.149]	slower	out	1.053 [1.033, 1.072]	slower	eq	0.904 [0.899, 0.908]	faster	inc
SQLite	1.084 [1.049, 1.132]	slower	inc	1.037 [0.995, 1.060]	n.e.	eq	1.044 [0.993, 1.134]	n.e.	inc	0.930 [0.928, 0.932]	faster	eq

TABLE 13. User-space instructions for one replay of each trace (millions, median of five replays) and the ratio of STRATA to the two conventional tables. The count of a table varied by at most 0.03 percent between replays.

Trace	STRATA	C_0	C_1	STRATA/C_0	STRATA/C_1
cjson	5.06	3.51	3.88	1.44	1.30
freertos	10.70	7.45	8.17	1.44	1.31
littlefs	5.24	3.55	3.74	1.48	1.40
lua	50.38	33.98	37.58	1.48	1.34
mbedtls	297.82	205.82	226.43	1.45	1.32
sqlite	58.63	43.35	44.46	1.35	1.32
tinycc	21.32	14.60	15.29	1.46	1.39
zlib	14.72	10.12	11.22	1.45	1.31

TABLE 14. Ablation: Change Relative to Default STRATA in Smallest Fixed Buffer and in Replay Time (Percent; Verdict From the 95 Percent Interval).

	Raw names		Sparse index		No shrinking
Project	Buffer	Time	Buffer	Time	Time
cJSON	+5.8	−0.2 (n.e.)	+26.7	−12.1 (faster)	−2.8 (faster)
littlefs	+7.5	+4.4 (n.e.)	+20.0	−12.9 (faster)	+0.7 (n.e.)
zlib	+6.0	+3.1 (n.e.)	+25.4	−9.0 (faster)	−4.9 (faster)
Lua	+6.1	+7.1 (slower)	+22.3	−11.8 (faster)	+1.4 (n.e.)
FreeRTOS	+8.5	+3.0 (n.e.)	+18.2	−9.3 (faster)	−4.4 (n.e.)
Mbed TLS	+9.9	−1.6 (n.e.)	+20.0	−4.8 (n.e.)	+2.7 (slower)
TinyCC	+7.5	+9.9 (slower)	+20.4	−9.2 (faster)	+1.2 (n.e.)
SQLite	+8.9	−0.2 (n.e.)	+22.8	−4.5 (n.e.)	+2.9 (n.e.)

are ordinary names under a key their author does not know. With the unkeyed hash and a probe limit of 64, time grows linearly, at the price that 32 704 of the 32 768 colliding names are refused and only 64 are admitted: the capped table finishes quickly because it does almost no useful work, and its time is not comparable with that of a table that stores all names. Flood and honest names were never mixed in one table, so the number of honest names that a capped table would also refuse next to such a flood was not measured. The limit bounds the work of a call whatever the hash does. A keyed hash defeats the collision family built against the unkeyed mixer; that is evidence about this one family, not a proof that no collision strategy works, and a fresh key is argued under the assumptions of Section VI-C. On the compiler traces the keyed hash increases replay time by 20.2 to 32.2 percent (Table 10).

I. THE REFUSAL LAW AGAINST THE IMPLEMENTATION AND THE TRACES

Each trial draws a fresh random 64-bit key, builds a table A without a probe limit and a table B with limit L (same key, same index size), offers both the same n random distinct names, and then offers one more fresh name. Table 17 and Fig. 12 compare the exact law with process (P2): the observed frequency, over *all* trials, that the probe count of the last name in A exceeds L, which is the event that B would refuse it after a refusal-free fill. All seven z-scores lie within ± 1.96 (largest $|z| = 1.96$). The *clean* trials are those in which B refused none of the n earlier names; they are not used for the observed rate. In them B was compared with A in lock step (equal state digests and B refuses the last name iff the probe count exceeds L), with no mismatch; this validates the equivalence of the limited and unlimited tables up to the first refusal, and says nothing about behavior after a refusal. That is the content of Table 18: for each setting it lists the trials with a refusal in the fill, the refused fill operations, the frequency of a refusal of the last name given a clean fill (P3), and the frequency of a refusal of the last name in B over all trials after it kept inserting past its refusals (P4). Both frequencies lie below the exact law in every setting (P3 by 28 to 51 percent, P4 by 2 to 17 percent), consistent with the bounds of this section, and in no trial did B refuse a name that A would have admitted within L probes ("violations" $= 0$), which is the inclusion used in the coupling argument. The settings have refusal probabilities between 4×10^{-4} and 6×10^{-2}, which 400 000 trials resolve; the design values near 10^{-6} and 10^{-11} are not tested by trials, they rest on the formula, its enumeration and independent-derivation checks, and the stored simulation of 240 000 000 insertions (Fig. 6). The law therefore describes the first capped query of the implementation, not only of the model; the refusal-filtered process is covered by the bounds, not by an exact law.

The compiler traces alone would have suggested a much smaller limit. Replaying the 15 compiler traces (719 833 declarations) in bounded mode, with the index sized for each trace's peak, a limit of 64 refuses no declaration with either hash and the longest probe sequence is 47 (Table 19). Yet at the load limit of $3/4$, 0.77 percent of fresh names need more than 64 probes: the traces' tables are small and spend most of their time well below the load limit. The exact law gives

TABLE 15. Synthetic Stress Traces: Replay Time of STRATA Relative to the Conventional Arena Table and to METIS-X (Percent, With Verdict), and Whether METIS-X Agrees With the Reference Model.

Trace	Declarations	Maximum depth	STRATA vs. arena table	STRATA vs. METIS-X	METIS-X result
`big_scope_exit`	200 100	1	−39.0 (faster)	−53.6 (faster)	agrees
`deep_nesting`	89 949	200	−12.1 (faster)	−32.1 (faster)	agrees
`deep_shadow_400`	400	400	−51.3 (faster)	−96.4 (faster)	149 refused
`dense_50k`	50 000	0	+0.7 (n.e.)	−26.8 (faster)	agrees
`long_ids`	89 899	10	+6.9 (n.e.)	−38.9 (faster)	agrees
`lookup_heavy`	15 097	10	+12.5 (slower)	−0.1 (n.e.)	agrees
`scope_churn`	150 050	1	−24.7 (faster)	−26.3 (faster)	agrees
`shadow_heavy`	134 903	40	−21.2 (faster)	−54.3 (faster)	agrees
`short_ids`	89 646	10	−16.3 (faster)	−36.0 (faster)	agrees

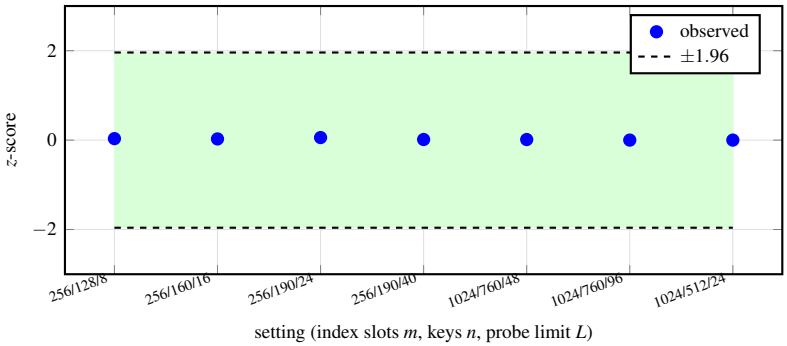


FIGURE 12. Agreement of the exact refusal law with the real bounded-mode table: standard scores in seven settings of 400 000 trials each, all within plus or minus 1.96.

TABLE 16. Declaring 32 768 Names of 150 Characters in Four Configurations: Time in Milliseconds, Their Ratio, Refused Names and Longest Probe Sequence for the Colliding Input.

Configuration	Ordinary	Colliding	Ratio	Refused	Longest
Default	6.52	1953.86	299.7	0	32 768
Keyed hash	9.33	10.80	1.2	0	106
Limit 64	3.11	7.72	2.5	32 704	64
Limit 64, keyed	5.22	5.56	1.1	0	26

TABLE 17. Process (P2): Exact Law Against the Real Probe-Limited Table. Observed Frequency Over All Trials That the Last Name Needs More Than *L* Probes, the Exact Value, the *z*-Score, Clean Trials (No Refusal in the Fill) and Lock-Step Mismatches in Clean Trials (*m* Index Slots, *n* Keys Stored, *L* Probe Limit).

m	n	L	trials	observed	exact	z	clean trials	mismatches
256	128	8	400 000	0.03311	0.03315	-0.13	254 871	0
256	160	16	400 000	0.02643	0.02667	-0.94	307 410	0
256	190	24	400 000	0.05740	0.05730	+0.26	247 303	0
256	190	40	400 000	0.01372	0.01375	-0.19	369 939	0
1024	760	48	400 000	0.01251	0.01286	-1.96	307 286	0
1024	760	96	400 000	0.00060	0.00058	+0.52	397 066	0
1024	512	24	400 000	0.00040	0.00046	-1.63	395 607	0

the setting directly: 258 for a refusal probability of 10^{-6} per declaration at 2^{16} slots, or 52 with an index kept at most half full. The limit of 512 recommended in the library documentation is safe but conservative: at 65 536 slots it corresponds to a refusal probability of 2.4×10^{-11} per declaration.

J. WHEN DOES THE SMALLER BUFFER MATTER?

A saving of one sixth decides whether a workload fits only when the budget lies between the needs of the two tables.

TABLE 18. Processes (P3) and (P4) in the Same 400 000 Trials per Setting: Clean Trials, Trials With a Refusal in the Fill, Refused Fill Operations, Frequency of a Refusal of the Last Name Given a Clean Fill (P3) and in the Table That Kept Inserting After Refusals (P4), and Trials in Which the Limited Table Refused a Name the Unlimited One Admitted Within *L* Probes.

m	n	L	clean	with refusal	refused in fill	(iii) $\hat{P}$	(iv) $\hat{P}$	violations
256	128	8	254 871	145 129	210 813	0.02384	0.02931	0
256	160	16	307 410	92 590	137 811	0.01664	0.02325	0
256	190	24	247 303	152 697	282 374	0.03151	0.04899	0
256	190	40	369 939	30 061	45 241	0.00810	0.01246	0
1024	760	48	307 286	92 714	169 972	0.00624	0.01109	0
1024	760	96	397 066	2 934	4 327	0.00040	0.00057	0
1024	512	24	395 607	4 393	5 212	0.00030	0.00038	0

TABLE 19. Bounded Mode on 15 Compiler Traces (719 833 Declarations): Declarations Refused by the Probe Limit and Number of Traces Affected, for the Default and the Keyed Hash.

	Default hash		Keyed hash	
Limit	Refused	Traces	Refused	Traces
4	15 335	15	15 589	15
8	3 543	15	3 507	15
16	561	12	571	11
32	51	5	51	4
64	0	0	0	0
128	0	0	0	0
256	0	0	0	0
512	0	0	0	0

Of the 169 translation units of the eight projects, the units that only STRATA can hold are few at every budget, at most 23 at 16 KiB (Table 20). On the emulated microcontroller (Section IX-L), three whole workloads fit a buffer in which

TABLE 20. Translation Units (of 169) That Fit a Fixed Budget: in Both Single-Buffer Tables, Only in STRATA, or in Neither.

Budget (KiB)	Both	Only STRATA	Neither
8	1	1	167
16	28	23	118
32	100	7	62
64	146	1	22
128	167	0	2

TABLE 21. Sweep Over the Shadowing Factor σ and the Name Length: Saving of STRATA in Smallest Fixed Buffer (Percent) and Replay Time Relative to the Conventional Arena Table (Percent).

	6 characters		12 characters		24 characters	
σ	Save	Time	Save	Time	Save	Time
1.0	0	−30.7 (faster)	0	+0.6 (n.e.)	9	+6.1 (n.e.)
1.5	24	−36.7 (faster)	25	+5.5 (n.e.)	33	+5.8 (slower)
2.0	29	−35.2 (faster)	31	+8.1 (slower)	41	+12.2 (slower)
4.0	43	−47.5 (faster)	47	−4.4 (n.e.)	57	+16.0 (slower)
8.0	50	−39.9 (faster)	55	+2.8 (n.e.)	65	+20.9 (slower)

the conventional table refuses operations. A sweep over the shadowing factor (live declarations per distinct name) and the name length confirms (1) and the ledger: the saving grows with shadowing, since a hidden declaration costs three words in STRATA and a full entry with the name in the conventional table, and reaches 65 percent at the highest factor tested (Table 21). Replay time relative to the conventional table follows the name length rather than the shadowing factor.

K. INSIDE A COMPILER

To see what the differences amount to in a whole compiler, we replaced the per-scope hash maps that chibicc [41] uses for variable scopes by STRATA and, separately, by the conventional arena table, and compiled seven of the projects with all three builds. The unmodified compiler accepts 158 of their 170 translation units; on those, the assembly produced with STRATA is byte-for-byte identical to the original on 158 units and with the arena table on 158. Over 21 interleaved rounds (Table 22), whole-project compile time with STRATA changes by between −3.7 and +2.2 percent, and with the arena table by between −3.1 and +4.3 percent; peak resident memory per unit changes by a median of 0.18 percent. In a hosted compiler neither single-buffer table matters: symbol-table work is a small part of the compile. With STRATA the interval excludes zero on one project, Mbed TLS, where it is slower by 1.4 percent (interval 0.2 to 2.4); the 14 intervals are not corrected for multiple comparisons, and the other medians are quoted in the range above. The experiment establishes that the tables are correct in a real resolution path; it does not show a benefit for compilers that are not memory-constrained.

L. AN EMULATED MICROCONTROLLER

We built a bare-metal image for a Cortex-M3 and ran it under QEMU with 64 KiB of memory, with allocation functions stubbed to fail. Five of the eight traces fit that memory; all 5 replay with a checksum over every result identical to the host's, using 492 bytes of stack, and the table's code occupies

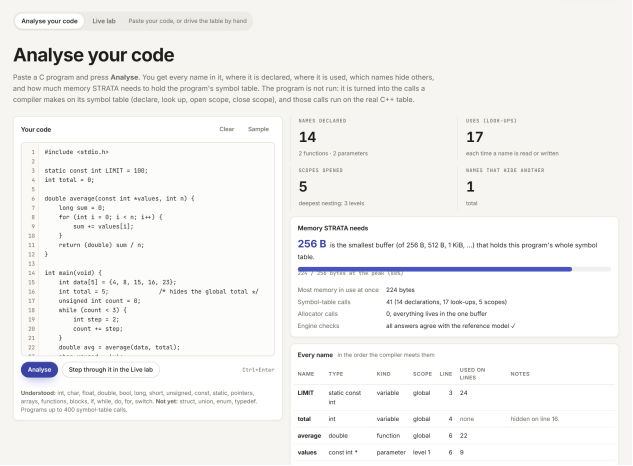


FIGURE 13. The "Analyse your code" page. A pasted C program is translated into the calls a compiler makes on its symbol table, which run on the real STRATA table in a local engine; the page lists every name with its type, scope, line and uses, the names it hides, unused and undeclared names, and the smallest buffer that holds the program.

2 715 bytes. The same 5 traces pass in bounded mode with the keyed hash and a probe limit of 512, with no refusal. At equal buffers, STRATA fits FreeRTOS in 48 KiB, Lua in 40 KiB and cJSON in 32 KiB, where the conventional table refuses 420, 2 214 and 822 operations. The 64 KiB limit is enforced by the linker script only, not by an emulator option; the trace input is constant data in flash and is not counted in it. The stack figure is the high-water mark of one run in fixed mode (492 bytes; 484 in bounded mode) measured by a painted stack region, and the code figure is the sum of the sizes of the `strata::` symbols in the unlinked object, not a linked image. The emulator provides no timing, and no physical device was used, so these results show that the code fits, runs and returns identical answers within the stated memory, not how fast it runs on a chip. We chose emulation because the claims of this article concern memory layout and operation counts, which do not depend on the silicon, and we report instruction counts (Section IX-E) in place of wall-clock time for the same reason; a timing on a physical device is not claimed.

X. AN INTERACTIVE TOOL

To make the design inspectable, the artifact includes a web interface (React [44] and Vite [45]) whose local server runs a C++ engine linked with the unmodified header. Every request replays the whole list of operations on a fresh table, so the C++ table is always the source of truth; after each request the engine checks the table's invariants, that every decoded record equals what `resolve` returns, and that every answer equals the reference model. The *Analyse your code* page (Fig. 13) accepts a C program, translates it into declare, lookup, enter and exit calls, and reports every name, its scope and uses, shadowing, and the smallest buffer from 256 B to 64 KiB that refuses nothing. A *Live lab* shows the buffer decoded word by word after each call and the digest before and after a refused

TABLE 22. End to End in chibicc: Whole-Project Compile Time With Variable Scopes on STRATA or on the Conventional Arena Table, Relative to the Original Compiler (Percent, 95 Percent Interval; Negative Is Faster).

Project	Units compiled	Original compiler (s)	STRATA		Arena table	
cJSON	2	0.11	−0.7 [−5.9, +8.8]	n.e.	+4.3 [−2.6, +10.7]	n.e.
littlefs	1	0.02	−3.7 [−5.8, +2.6]	n.e.	−3.1 [−6.4, −0.7]	faster
zlib	12	0.30	+2.2 [−3.0, +5.7]	n.e.	+0.5 [−3.9, +4.9]	n.e.
Lua	34	1.09	−0.7 [−3.5, +0.5]	n.e.	−1.8 [−2.7, −0.3]	faster
FreeRTOS	7	0.65	+1.8 [−4.1, +5.5]	n.e.	−0.5 [−7.0, +7.8]	n.e.
Mbed TLS	101	5.58	+1.4 [+0.2, +2.4]	slower	+1.0 [−0.7, +1.5]	n.e.
SQLite	1	1.00	+0.0 [−3.3, +2.8]	n.e.	−2.1 [−3.6, −0.2]	faster

call, and the results and theory pages draw every figure from the stored measurements and recompute the refusal law in the browser. The interface changes neither the library nor any result; its 37 automated checks compare it with the engine and with the Python programs of the analysis.

XI. DISCUSSION

What the compact design buys. Against a conventional chained table that obeys the same storage discipline and stores a hash word, STRATA offers a buffer about one sixth smaller; against the same table without the hash word, which we implemented and measured, the buffer is 5.3 to 9.0 percent smaller, all of it from name packing. In the first run no difference in speed was established on most traces; in five repeated sessions on another configuration STRATA was slower than both conventional tables (8 to 20 percent against the one with the hash word). A conventional table can obtain most of the saving by dropping its stored hash, at a cost in time of its own. For a compiler on a workstation neither table matters. For a front end with tens of kilobytes, one sixth of the symbol table is memory that can go elsewhere, and on the emulated microcontroller it decided whether three workloads fit.

What the storage discipline buys. One buffer and write-after-reserve give a refusal that is simple to state and simple to test exhaustively at small scale. We consider this the more transferable result; it applies equally to the conventional table.

Why last-in-first-out removal matters beyond speed. The removal rule was adopted to avoid tombstones. Its more useful consequence is Theorem 1: because the table is always an insertion-only table, the classical exact analysis of linear probing applies to it at every moment, and Theorem 2 shows that no other deletion order would allow this for every table.

Why measurement alone was not enough. Three times in this work a measurement supported a conclusion that analysis overturned: the time penalty of Section IX-F, the probe limit of 64 that the traces suggested, and the reading of the buffer saving as a benefit of interning. These are arguments for pairing replay experiments with a baseline that shares the design's constraints and with an exact analysis of the quantity being tuned.

Where the design is the wrong choice. If memory is not constrained and the input is trusted, this study gives no reason to prefer a single-buffer table. If the input is untrusted, the keyed hash costs 20.2 to 32.2 percent of replay time, and its guarantee requires a fresh key per table.

XII. LIMITATIONS AND THREATS TO VALIDITY

Mostly replay, not compilation. The traces record what a compiler's own tables were asked to do; replaying them measures a table, not a compiler built on it. The one experiment inside a compiler replaces variable scopes only, on a hosted machine, and shows no end-to-end benefit.

Two machine configurations, no hardware. Timing comes from two configurations, a laptop under a hypervisor and a shared native desktop, and the microcontroller is emulated. The verdicts on individual traces are not stable across runs: in an earlier run of the same code STRATA was faster than the conventional table on two traces and slower on three, and in the repeated sessions it was slower on all eight. The sessions are not on an isolated machine (frequency scaling, a lightly loaded sibling thread). The traces are correlated, the intervals are exploratory and uncorrected for multiplicity, and the equivalence margin of ± 10 percent is a choice. The only claim the timing supports is that the compact table is not faster than a conventional single-buffer table.

Workloads. The traces are C only, from two small compilers, and depend on the host's system headers. Names outside the identifier alphabet, as in mangled C++ names, take the unpacked path, which the traces exercise little.

Baselines. The conventional arena table and four further baselines were written by the authors of the table under study; they are not independent implementations. LLVM's and Clang's own containers were not measured.

The analysis. The proofs are pen-and-paper, checked by exhaustive enumeration and by comparison with the implementation, not by a proof assistant. The refusal law assumes an ideal hash; against an adversary it assumes that HalfSipHash with one and three rounds and a 64-bit key behaves as one and that keys are fresh. The key-lifetime experiment uses an idealized oracle; no timing channel was measured. A limit law for refusals on wide and deep scope trees is open.

Design limits. Names of at most 255 bytes, 16 383 nested scopes, a buffer of at most 64 MiB, one name space per table, single-threaded use.

Literature. The search for related work was limited, and several sources were consulted at the level of abstracts or documentation.

XIII. CONCLUSION

A scoped symbol table can be confined to one fixed buffer, refuse cleanly when the buffer is full, and, in bounded mode,

bound the index slots examined by each declaration and lookup on hostile input (scope exit and the default-mode rebuild are not bounded this way), at a cost that we have measured and with a refusal probability that we can compute in the ideal-hash model. On events logged by a C compiler for eight projects, the compact design studied here needs a buffer about one sixth smaller than a conventional single-buffer table, a saving that an exact ledger attributes to a stored hash word and name packing, a saving that falls to between 5 and 9 percent against a conventional table without a stored hash word, which we implemented and measured. In time it is not faster: slower than both conventional tables in repeated sessions on one configuration, with no difference established on most traces in an earlier run. Both are well ahead of a multi-allocation predecessor, and that larger gain belongs to the storage discipline. Because declarations are removed in the reverse order of their creation, the table is always an insertion-only table, which is the only deletion order with this property for every table and which makes the refusal probability of a probe limit exactly computable: at load three quarters, a one-in-a-million refusal needs 258 probes, not the 455 a tail bound suggests, and the law agrees with the implementation. Identifiers that collide for every seed of the default hash slow declarations 300-fold, and a keyed hash removes the effect for that collision family; under the stated assumptions a fresh key per table keeps the refusal law valid against an oblivious adversary.

The most useful next steps are a compiler that runs on the table on a physical microcontroller, workloads from languages other than C, a limit law for refusals on deep scope trees, and machine-checked proofs.

APPENDIX A PROOF OF THE EXACT REFUSAL LAW AND OF THE BOUNDS FOR THE REFUSAL-FILTERED PROCESS

Let $k = m-n \geq 1$. For $k = 1$ one cell is empty and uniformly located, and the L cells from a uniform start are occupied iff the start lies among the $(m-L)^+$ cells that precede the empty cell by at least L. Assume $k \geq 2$ and let X_j be the number of keys with home j; (X_j) is multinomial with probability proportional to $\prod_j 1/x_j!$.

Step 1: blocks. Since $n < m$ there are exactly k empty cells, and the occupied set does not depend on the insertion order [46]. Cut the ring at the empty cells: a *block* is an empty cell together with the occupied cells after it up to the next empty cell, so a block of g cells has $g-1$ occupied ones. No key crosses an empty cell, so the keys homed in a block occupy exactly its cells. With home counts y_u of the $g-1$ occupied cells, the cells are occupied and the next one is empty iff $\sum_{t\leq u} y_t \geq u$ for $u < g-1$ and $\sum_{t\leq g-1} y_t = g-1$. Conversely, a cyclic concatenation of such blocks has exactly these empty cells, so home-count vectors correspond bijectively to placements of k blocks around the ring.

Step 2: weight of a block. Put $y_g = 0$ and $z_t = y_t - 1 \geq -1$. The condition says that the walk $z_1+\cdots+z_u$ stays nonnegative for $u < g$ and first reaches -1 at $u = g$. By the cycle lemma [37], exactly one of the g cyclic rotations of a sequence in $\mathbb{N}^g$ with sum $g-1$ has this property. The weight $\prod_t 1/y_t!$ is rotation invariant and $\sum_{\sum y=g-1} \prod_t 1/y_t! = g^{g-1}/(g-1)!$ by the multinomial theorem, so the total weight of a block of g cells is $B(g) = g^{g-1}/g!$.

Step 3: the block of a cell. The weight of a ring configuration is the product of its block weights. Cell 0 lies in a block of g cells at one of g positions; the remaining $k-1$ blocks contain $m-g$ cells with total weight $Q_{k-1}(m-g)$, where $Q_j(s) = \sum_{g_1+\cdots+g_j=s} \prod B(g_i) = \frac{j}{s}\, s^{s-j}/(s-j)!$ by the Borel–Tanner identity [38]. By exchangeability, $\sum_g g\, B(g) Q_{k-1}(m-g) = \frac{m}{k} Q_k(m)$, hence cell 0 lies in a block of g cells with probability $\frac{k}{m}\, g\, \pi_g$, $\pi_g = B(g)Q_{k-1}(m-g)/Q_k(m)$, at a uniform position.

Step 4: distance to the next empty cell. For position $o \in \{2,\dots,g\}$ of a block of g cells the next empty cell is $g+1-o$ steps away, so the L cells from a start are all occupied iff $o \leq g+1-L$, which holds for $(g-L)^+$ positions; position 1 is empty. Therefore the probability that the first L cells from an independent uniform home are occupied is $\sum_g \frac{k}{m} g\, \pi_g\, (g-L)^+/g = \frac{k}{m} \sum_{g>L} (g-L)\pi_g$, with $g \leq n+1$.

Bounds for (P3) and (P4). (P3): $\mathbb{P}(\text{refused} \mid \text{clean}) = \mathbb{P}(\text{refused} \wedge \text{clean})/\mathbb{P}(\text{clean}) \leq R_m(n,L)/P_{\text{fill}}$, and $P_{\text{fill}} \geq 1 - \sum_{t<n} R_m(t,L)$, because the event that some push of the fill is refused at limit L is the event that some push of the unlimited fill has probe count above L, a union of events of probability $R_m(t,L)$. (P4), insertions only: let A be the unlimited table and B the limited one, both receiving the same fresh names; we show $\mathrm{occ}(B) \subseteq \mathrm{occ}(A)$ by induction. If B refuses a name, B is unchanged and A only adds a cell. If B accepts a name at cell c_B, the cells of its probe path before c_B are occupied in B and therefore in A, so A places the name at a cell c_A at or after c_B on the path, and $c_B \in \mathrm{occ}(A) \cup \{c_A\}$. If B refuses a name, its first L cells are occupied in B and therefore in A; so B refuses only if the probe count in A exceeds L. The home of the offered name is independent of the table, hence the probability of refusal at the tth offer is at most $R_m(t-1,L)$.

DATA AND CODE AVAILABILITY

The implementation, the tests, the compiler patches, the traces, the raw measurements of every run cited here, the verification programs of the analysis, the interactive tool, and the scripts that regenerate every number, table and figure of this article are public [36]. The artifact contains a manifest that maps every table, figure and headline number of this article to the command that produces it, with expected run times (`docs/REVIEW_MANIFEST.md`), SHA-256 checksums of every trace file, the raw paired rounds and per-session context of Section IX-E (`results/review/`), and the commit and the git tag `pre-cleanup` of the state before the repository was cleaned; the exact release to cite is fixed when the article is accepted. The enumeration of Section VII needs about 3.5 minutes, the refusal-law check against the real table about 5 minutes, the timing sessions about one hour including gaps, and the full benchmark pipeline several hours on one core. The formula of Theorem 3 is implemented in exact rational arithmetic (Python, compared with brute force for $m \leq 9$) and in floating point (Python and TypeScript);

the TypeScript version is checked against the Python values, which is a transcription check and not an independent derivation.

ACKNOWLEDGMENT

Funding. This work received no external funding. **Competing interests.** The authors declare no competing interests. **Author contributions.** A. Jain and S. Singh contributed equally to the work. Prof. Bhuvaneshwari M. guided the project throughout and is the corresponding author.

In accordance with the journal's policy on the disclosure of generated content: artificial-intelligence assistants (Claude, Anthropic) were used, at the direction of the authors, for planning the work, for assistance in writing the code and the verification programs, for help with the literature review, and for drafting and editing parts of the text. The tool is not an author. Every number is produced by programs that can be re-run; the proofs are pen-and-paper and were checked by exhaustive enumeration and by statistical comparison with the implementation, not by a proof assistant. All authors have read, checked and verified the content and take responsibility for all of it. A fuller account is in the repository (`docs/AI_USE_DISCLOSURE.md`).

REFERENCES

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.

[2] S. L. Graham, W. N. Joy, and O. Roubine, "Hashed symbol tables for languages with explicit scope control," in *Proc. SIGPLAN Symposium on Compiler Construction*, 1979, pp. 50–57.

[3] A. W. Appel, *Modern Compiler Implementation in C*. Cambridge, U.K.: Cambridge University Press, 1998.

[4] S. A. Crosby and D. S. Wallach, "Denial of service via algorithmic complexity attacks," in *Proc. 12th USENIX Security Symposium*, 2003, [Online]. Available: https://www.usenix.org/conference/12th-usenix-security-symposium/denial-service-algorithmic-complexity-attacks.

[5] J.-P. Aumasson and D. J. Bernstein, "SipHash: A fast short-input PRF," in *Progress in Cryptology – INDOCRYPT 2012*, ser. Lecture Notes in Computer Science, vol. 7668. Springer, 2012, pp. 489–508.

[6] LLVM Project, "ScopedHashTable.h, LLVM ADT library," [Online]. Available: https://llvm.org/doxygen/ScopedHashTable_8h_source.html. Accessed: Oct. 13, 2026.

[7] F. Bellard and TinyCC contributors, "Tiny C Compiler, commit 43c7708," [Online]. Available: https://repo.or.cz/tinycc.git. Accessed: Oct. 11, 2026.

[8] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1998.

[9] M. A. Bender, B. C. Kuszmaul, and W. Kuszmaul, "Linear probing revisited: Tombstones mark the demise of primary clustering," in *Proc. IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS)*, 2021, pp. 1171–1182.

[10] M. Farach-Colton, A. Krapivin, and W. Kuszmaul, "Optimal bounds for open addressing without reordering," in *Proc. 65th IEEE Symposium on Foundations of Computer Science (FOCS)*, 2024, pp. 594–605, statements used here were read in the arXiv version 2501.02305.

[11] C. W. Fraser and D. R. Hanson, *A Retargetable C Compiler: Design and Implementation*. Redwood City, CA: Addison-Wesley, 1995, iSBN 0-8053-1670-1.

[12] D. R. Hanson, "Fast allocation and deallocation of memory based on object lifetimes," *Software: Practice and Experience*, vol. 20, no. 1, pp. 5–12, 1990.

[13] MicroPython Project, "MicroPython string interning," [Online]. Available: https://docs.micropython.org/en/latest/develop/qstr.html. Accessed: Oct. 13, 2026.

[14] R. M. Jiménez and C. Martínez, "On deletions in open addressing hashing," in *Proc. 15th Workshop on Analytic Algorithmics and Combinatorics (ANALCO)*, 2018, pp. 23–31.

[15] P. Sanders, "Hashing with linear probing and referential integrity," arXiv:1808.04602, 2018, full text read; it contains no statement of stack-ordered deletion.

[16] P. Celis, P.-Å. Larson, and J. I. Munro, "Robin Hood hashing," in *Proc. 26th Annual Symposium on Foundations of Computer Science (SFCS)*, 1985, pp. 281–288.

[17] A. Jain and S. Singh, "METIS: Symbol-table research repository, commit 7b3dacf," 2026, [Online]. Available: https://github.com/adhyan-jain/Metis. Accessed: Oct. 11, 2026.

[18] R. Pagh and F. F. Rodler, "Cuckoo hashing," *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.

[19] M. Herlihy, N. Shavit, and M. Tzafrir, "Hopscotch hashing," in *Distributed Computing*, ser. Lecture Notes in Computer Science. Springer, 2008, pp. 350–364.

[20] M. Naor and V. Teague, "Anti-persistence: History independent data structures," in *Proc. 33rd Annual ACM Symposium on Theory of Computing (STOC)*, 2001, pp. 492–501.

[21] G. E. Blelloch and D. Golovin, "Strongly history-independent hashing with applications," in *Proc. 48th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 2007, pp. 272–282.

[22] P. Flajolet, P. Poblete, and A. Viola, "On the analysis of linear probing hashing," *Algorithmica*, vol. 22, no. 4, pp. 490–515, 1998.

[23] S. Janson, "Asymptotic distribution for the cost of linear probing hashing," *Random Structures & Algorithms*, vol. 19, no. 3–4, pp. 438–471, 2001.

[24] S. Janson and A. Viola, "A unified approach to linear probing hashing with buckets," *Algorithmica*, vol. 75, no. 4, pp. 724–781, 2016, section 11 read in the arXiv version 1410.5967.

[25] S. Janson, "Individual displacements for linear probing hashing with different insertion policies," *ACM Transactions on Algorithms*, vol. 1, no. 2, pp. 177–213, 2005.

[26] A. Viola, "Exact distribution of individual displacements in linear probing hashing," *ACM Transactions on Algorithms*, vol. 1, no. 2, pp. 214–242, 2005.

[27] M. A. Bender, A. Conway, M. Farach-Colton, W. Kuszmaul, and G. Tagliavini, "Tiny pointers," *ACM Transactions on Algorithms*, vol. 21, no. 4, pp. 1–43, 2025, theorem numbers refer to the arXiv version 2111.12800, which was read.

[28] A. Alharbi, C. Banderier, and C. Bouillaguet, "Bounded linear probing hashing," in *37th International Conference on Probabilistic, Combinatorial and Asymptotic Methods for the Analysis of Algorithms (AofA 2026)*, ser. LIPIcs, vol. 381, 2026, pp. 28:1–28:17, abstract, introduction, Section 3 and conclusion read.

[29] A. Pagh, R. Pagh, and M. Ružić, "Linear probing with constant independence," in *Proc. 39th Annual ACM Symposium on Theory of Computing (STOC)*, 2007, pp. 318–327.

[30] M. Thorup, "Linear probing with 5-independent hashing," arXiv:1509.04549, 2015, [Online]. Available: https://arxiv.org/abs/1509.04549.

[31] J.-P. Aumasson, D. J. Bernstein, and M. Boßlet, "Hash-flooding DoS reloaded: Attacks and defenses," Presentation slides, 2012, [Online]. Available: https://www.aumasson.jp/siphash/siphashdos_appsec12_slides.pdf.

[32] J. A. Donenfeld, "SipHash – a short input PRF," Linux kernel documentation, [Online]. Available: https://docs.kernel.org/security/siphash.html. Accessed: Oct. 13, 2026.

[33] R. J. Lipton and J. F. Naughton, "Clocked adversaries for hashing," *Algorithmica*, vol. 9, no. 3, pp. 239–252, 1993.

[34] M. Naor and E. Yogev, "Bloom filters in adversarial environments," in *Advances in Cryptology – CRYPTO 2015*, ser. Lecture Notes in Computer Science. Springer, 2015, pp. 565–584.

[35] Rust Project Developers, "rustc-hash: A speedy, non-cryptographic hashing algorithm used by rustc," [Online]. Available: https://github.com/rust-lang/rustc-hash. Accessed: Oct. 13, 2026.

[36] S. Singh, A. Jain, A. Venkatataramana, and Arjuman, "STRATA: Source code, traces, and measurements," 2026, [Online]. Available: https://github.com/shubhisingh1510/cdesproject.

[37] M. Dwass, "The total progeny in a branching process and a related random walk," *Journal of Applied Probability*, vol. 6, no. 3, pp. 682–686, 1969.

[38] J. C. Tanner, "A derivation of the Borel distribution," *Biometrika*, vol. 48, no. 1–2, pp. 222–224, 1961.

[39] A. Appleby, "MurmurHash3, in SMHasher," [Online]. Available: https://github.com/aappleby/smhasher. Accessed: Oct. 13, 2026.
[40] J.-P. Aumasson, "SipHash and HalfSipHash reference implementation," [Online]. Available: https://github.com/veorq/SipHash. Accessed: Oct. 13, 2026.
[41] R. Ueyama, "chibicc: A small C compiler, commit 90d1f7f," [Online]. Available: https://github.com/rui314/chibicc. Accessed: Oct. 11, 2026.
[42] G. Popovitch, "The parallel hashmap, version 1.4.1," [Online]. Available: https://github.com/greg7mdp/parallel-hashmap. Accessed: Oct. 13, 2026.
[43] F. Bellard, "QEMU, a fast and portable dynamic translator," in *Proc. USENIX Annual Technical Conference, FREENIX Track*, 2005, pp. 41–46.
[44] Meta Open Source, "React," [Online]. Available: https://react.dev. Accessed: Oct. 20, 2026.
[45] Vite contributors, "Vite: Next generation frontend tooling," [Online]. Available: https://vite.dev/guide/. Accessed: Oct. 20, 2026.
[46] A. G. Konheim and B. Weiss, "An occupancy discipline and applications," *SIAM Journal on Applied Mathematics*, vol. 14, no. 6, pp. 1266–1274, 1966.

ADHYAN JAIN is an undergraduate student with the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India. Research interests include compilers, hashing and the analysis of algorithms.

SHUBHI SINGH is an undergraduate student with the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India. Research interests include compilers, data structures for constrained systems, and systems software.

BHUVANESHWARI M. is with the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India.